

ADC-Aware Quantization of Large Language Models for In-Memory Computing

by

Alina Burykina

Master Thesis in Computer Science and Software Engineering

Submission: May 19, 2026

Supervisor: Prof. Dr. Dmitry Vetrov

Statutory Declaration

Family Name, Given/First Name	Burykina, Alina
Matriculation number	30008248
Kind of thesis submitted	Master Thesis

English: Declaration of Authorship

I hereby declare that the thesis submitted was created and written solely by myself without any external support. Any sources, direct or indirect, are marked as such. I am aware of the fact that the contents of the thesis in digital form may be revised with regard to usage of unauthorized aid as well as whether the whole or parts of it may be identified as plagiarism. I do agree my work to be entered into a database for it to be compared with existing sources, where it will remain in order to enable further comparisons with future theses. This does not grant any rights of reproduction and usage, however.

This document was neither presented to any other examination board nor has it been published.

German: Erklärung der Autorenschaft (Urheberschaft)

Ich erkläre hiermit, dass die vorliegende Arbeit ohne fremde Hilfe ausschließlich von mir erstellt und geschrieben worden ist. Jedwede verwendeten Quellen, direkter oder indirekter Art, sind als solche kenntlich gemacht worden. Mir ist die Tatsache bewusst, dass der Inhalt der Thesis in digitaler Form geprüft werden kann im Hinblick darauf, ob es sich ganz oder in Teilen um ein Plagiat handelt. Ich bin damit einverstanden, dass meine Arbeit in einer Datenbank eingegeben werden kann, um mit bereits bestehenden Quellen verglichen zu werden und dort auch verbleibt, um mit zukünftigen Arbeiten verglichen werden zu können. Dies berechtigt jedoch nicht zur Verwendung oder Vervielfältigung.

Diese Arbeit wurde noch keiner anderen Prüfungsbehörde vorgelegt noch wurde sie bisher veröffentlicht.

.....
Date, Signature

Abstract

Analog optical matrix-vector accelerators promise energy-efficient inference for large language models (LLMs), but they expose a failure mode absent from standard digital deployment: after each analog accumulation, a finite-resolution analog-to-digital converter (ADC) maps the output onto a hardware-fixed quantization lattice. This post-accumulation quantizer is especially damaging for large language models, whose channel-wise outliers make a single ADC scale difficult to use. Previous hardware-aware approaches commonly rely on retraining through the hardware model; for billion-parameter large language models, such quantization-aware training is often impractical because the original training data and compute budget are unavailable.

This thesis instead investigates whether an already trained large language model can be adapted in the post-training setting. Building on FlatQuant, an ADC-aware post-training quantization (PTQ) pipeline is introduced in which calibration is made to match analog inference by propagating ADC-quantized activations across blocks, mixing clean and ADC-aware objectives, and applying bounded diagonal scaling with staged calibration. On Llama-3.2-1B in the 4-bit weight and 4-bit activation (W4A4) setting, standard FlatQuant remains usable when the ADC is disabled, with WikiText-2 perplexity 15.41, but collapses when the ADC is enabled, reaching 2354. ADC-aware post-training quantization reduces this to 26.66, showing that transform-based calibration is necessary but plateaus far from the bfloat16 baseline of 8.68.

To close the remaining gap without full quantization-aware training, a rank-4 digital residual branch is added after the ADC, and only this low-rank correction is trained using cross-entropy and teacher Kullback–Leibler distillation. This 0.22%-parameter adaptation reduces WikiText-2/C4 ADC perplexity from 26.66/45.62 to **14.03/23.20**. These results show that fixed low-resolution ADCs require algorithmic adaptation beyond digital post-training quantization, and that lightweight output-side digital correction can recover much of the accuracy lost at the analog-digital boundary.

Contents

Abstract	iii
1 Introduction	1
2 Background and Related Work	3
2.1 In-Memory Computing and ADC-Constrained Inference	3
2.2 Quantization of Neural Networks	6
2.3 Challenges of Quantizing Transformer Models	7
2.4 Existing Methods	9
2.5 Research Gap	11
3 Problem Statement and Research Questions	12
3.1 Problem Definition	12
3.2 Research Questions	14
3.3 Scope and Assumptions	15
3.4 Evaluation Criteria	16
4 Proposed Method	17
4.1 Post-Training Quantization	17
4.2 Quantization-Aware Adaptation via Post-ADC Residual LoRA	22
5 Experiments and Results	23
5.1 Model, Data, and Hardware Setup	23
5.2 Evaluation Protocol	24
5.3 Main Results	24
5.4 Signed Readout: Ablation Results	27
5.5 Unsigned Shift-Subtract Readout: A Second Hardware Setting	37
6 Discussion	42
6.1 ADC Error Is Not the Same as Digital Quantization Error	42
6.2 ADC-Aware Calibration Reduces a Distribution Mismatch	42
6.3 Outliers Turn ADC Quantization into a Range-Allocation Problem	43
6.4 Why Pure PTQ Has a Natural Ceiling	43
6.5 Post-ADC Correction Works Because It Is Placed After the Lossy Step	43
6.6 Hardware Setting Matters	44
6.7 Practical Implications	44
6.8 Limitations	44
7 Conclusion	45
A Key Code Listings	51
A.1 ADC Forward Pass	51
A.2 Kronecker Transform with Diagonal	51
A.3 Block-Level Loss with Alpha-Mixing	52
A.4 Post-ADC Residual LoRA	53

1 Introduction

Large language models (LLMs) have become the dominant architecture for modern natural language processing (NLP), but their inference cost remains a major deployment obstacle. Transformer decoders are built primarily from large matrix–vector and matrix–matrix multiplications, and serving them requires repeatedly moving weights and activations between memory and compute units. In fact, autoregressive decoding is already memory-bandwidth-bound on modern GPUs: each forward pass reads billions of parameters to produce only a single output token, so the arithmetic intensity falls far below the hardware roofline and throughput is limited by memory bandwidth rather than compute [28]. This motivates hardware architectures that mitigate the data movement bottleneck by moving the dominant matrix–vector multiplications closer to memory, including in-memory computing (IMC) accelerators, where matrix–vector multiplication can be performed directly inside memory arrays rather than in a conventional digital compute core [39, 20, 5].

IMC is attractive for LLM inference because transformer layers are dominated by linear projections. In an idealized IMC pipeline, the weight matrix of a projection is stored in a memory array, an input activation vector is applied to the array, and the analog accumulation implements the dot product. However, this analog result must be converted back into a digital value before it can be consumed by the next layer, placing the analog-to-digital converter (ADC) directly on the neural network inference path. The precision and range of the ADC therefore determine how much information from the analog accumulation is preserved.

This thesis studies the interaction between transformer quantization and ADC-constrained analog inference. In a conventional digital quantization setting, the main sources of error are weight and activation quantization. In an IMC accelerator there is an additional output quantization stage: the accumulated analog result is passed through the ADC, which quantizes it to a fixed step size Δ and clips it to the ADC output range. The central difference from digital low-bit inference is that even if weights and activations are quantized well, the accumulated output can still be severely distorted by a coarse ADC lattice.

Unlike the activation and weight scales, which are adjusted during calibration, the ADC step size Δ is fixed by the analog hardware at design time and cannot be reduced by retraining or by changing the learned quantization transform. The only software-side levers available are how the signal is shaped before the ADC and how the quantized output is corrected after it. This makes ADC-constrained inference a different optimization problem from standard digital PTQ.

LLMs are a particularly challenging target for this setting. Prior work has shown that transformer activations contain large outlier channels, and that these outliers are a major obstacle to low-bit quantization [11, 38, 35, 2, 24]. Because activation scales are computed from the per-token maximum, a single large channel sets the scale for the entire activation vector. Typical activation values then receive very small integer codes, reducing the effective density of the integer dot product. In an ADC pipeline this can produce poorly occupied ADC bins or dead regions where small accumulated outputs collapse to zero. LLM activation outliers therefore affect both activation quantization and the quality of the analog output seen by the ADC.

This thesis focuses on decoder-only transformer LLMs, using Llama-3.2-1B as the main experimental target. The goal is to study whether transform-based post-training quantiza-

tion (PTQ), combined with ADC-aware calibration and lightweight adaptation, can make low-bit LLaMA inference viable under ADC-constrained analog computation. The work starts from FlatQuant, a transform-based PTQ method that learns layer-wise affine transformations to make weights and activations easier to quantize [23]. FlatQuant changes the coordinate system in which quantization is applied while leaving the linear map unchanged. The transformed activation distribution is flatter, so per-token scaling is no longer dominated by a few outlier channels.

Standard PTQ methods optimize for digital quantization and do not account for ADC output discretization as a separate source of error. Even with well-calibrated weights and activations, ADC-path perplexity remains substantially higher than no-ADC perplexity, the quality achievable with digital quantization alone. Closing this gap requires calibration that is explicitly sensitive to ADC noise and, ultimately, a lightweight correction stage applied after the ADC path. This thesis makes the following contributions:

1. ADC-constrained formulation for LLaMA PTQ. The thesis formulates LLaMA inference under a tiled ADC-constrained matrix–vector multiplication (MVM) model and implements a simulation pipeline with per-token activation quantization, per-channel weight quantization, integer accumulation, ADC output quantization, and dequantization.
2. Diagnosis of ADC-specific failure modes. The thesis separates ordinary weight and activation quantization error from ADC output quantization error using no-ADC evaluation, dead-rate, ADC bin usage, reconstruction error, and layer/projection ablations. This shows that dead zones can be caused by poor transforms, but that the remaining degradation can be dominated by ADC output resolution.
3. ADC-aware FlatQuant calibration. The thesis adapts FlatQuant to ADC-constrained inference by adding ADC propagation across blocks, an α -mixing objective between clean and ADC-propagated reconstruction, and a staged calibration schedule. For per-channel scaling, the method builds on the learnable scaling component already used in FlatQuant-style PTQ but studies its bounded form, placement between attention and MLP sub-layers, and interaction with ADC-aware calibration. Together these adaptations reduce INT4 ADC perplexity from 2354 (naive baseline) to 26.66 on WikiText-2.
4. Analysis of pure PTQ limitations. Extensive ablations show that additional dead-zone penalties, aggressive PACT-style clipping, fixed Hadamard initialization, and bin-center losses are insufficient or harmful in the studied setting. Pure PTQ improves substantially but eventually reaches a plateau beyond which further calibration yields diminishing returns.
5. Residual post-ADC low-rank correction. The thesis introduces a lightweight residual correction stage on top of the frozen quantized ADC path, trained with distillation against a full-precision teacher. With only 0.22% extra parameters, this stage drops INT4 ADC perplexity from 26.66 to 14.03 on WikiText-2 and from 45.62 to 23.20 on C4, while keeping the base ADC model frozen.

2 Background and Related Work

2.1 In-Memory Computing and ADC-Constrained Inference

In-memory computing (IMC) executes matrix–vector multiplication (MVM) directly inside memory arrays, reducing the cost of data movement that dominates conventional digital inference pipelines [39, 20, 5]. In crossbar-style accelerators, neural network weights are mapped to cell conductances, the input vector is applied as an analog excitation, and the array physics performs the accumulation. This is attractive for large neural networks because MVM is the dominant operation in both attention and MLP blocks. However, the result of the array is analog, whereas the next layer expects a digital tensor. Therefore, each analog MVM must be followed by analog-to-digital conversion (ADC).

The same analog-to-digital bottleneck appears across very different physical substrates. Memristive and phase-change-memory crossbars encode weights as cell conductances and exploit Ohm’s law plus Kirchhoff’s current summation for the multiply–accumulate [39, 20, 5]; photonic systems instead encode weights and activations as light intensities or phases and perform the MVM through optical interference, reading the result with photodetectors [33]. Despite the different physics, the common feature is that the analog accumulation is then sampled by a finite-resolution ADC before the next layer can consume it. Figure 1 illustrates this for the optical case: the analog signal carrying the MVM result is converted to a digital code through a photodetector front-end followed by an ADC, after which it re-enters the digital data path.

A simplified ADC-constrained inference pipeline, illustrated in Figure 4, follows the math model adopted by prior work on analog and ADC-constrained inference [34] and can be written as

$$x_q = Q_x(x), \quad (1)$$

$$W_q = Q_w(W), \quad (2)$$

$$y_{\text{int}} = W_q x_q, \quad (3)$$

$$y_{\text{adc}} = Q_{\text{ADC}}(y_{\text{int}}), \quad (4)$$

$$\hat{y} = D_{\text{out}} y_{\text{adc}}, \quad (5)$$

where Q_x and Q_w are the activation and weight quantizers, Q_{ADC} denotes the output quantizer implemented by the ADC, and D_{out} is the dequantization map. In the ADC model used throughout this thesis, the analog accumulation is quantized by a coarse output lattice:

$$Q_{\text{ADC}}(u) = \Delta \cdot \text{clip}\left(\left\lfloor \frac{u}{\Delta} \right\rfloor, n_a, p_a\right), \quad (6)$$

where Δ is the ADC step size and (n_a, p_a) are the output code bounds. In a tiled IMC setting, one common hardware model yields

$$\Delta = \frac{2 n_t q_x q_w}{2^{b_a} k}, \quad (7)$$

where n_t is the tile size, b_a is the ADC bitwidth, k is the ADC scaling factor, and q_x, q_w are the maximum activation and weight code levels. Equation (7) makes the hardware trade-off explicit: larger tiles or lower ADC precision lead to coarser output quantization.

The scaling factor k controls the trade-off between ADC resolution and clipping. The worst-case range of the integer accumulation is $[-n_t q_x q_w, +n_t q_x q_w]$, and the ADC covers a fraction $1/k$ of this range with 2^{b_a} uniform bins.

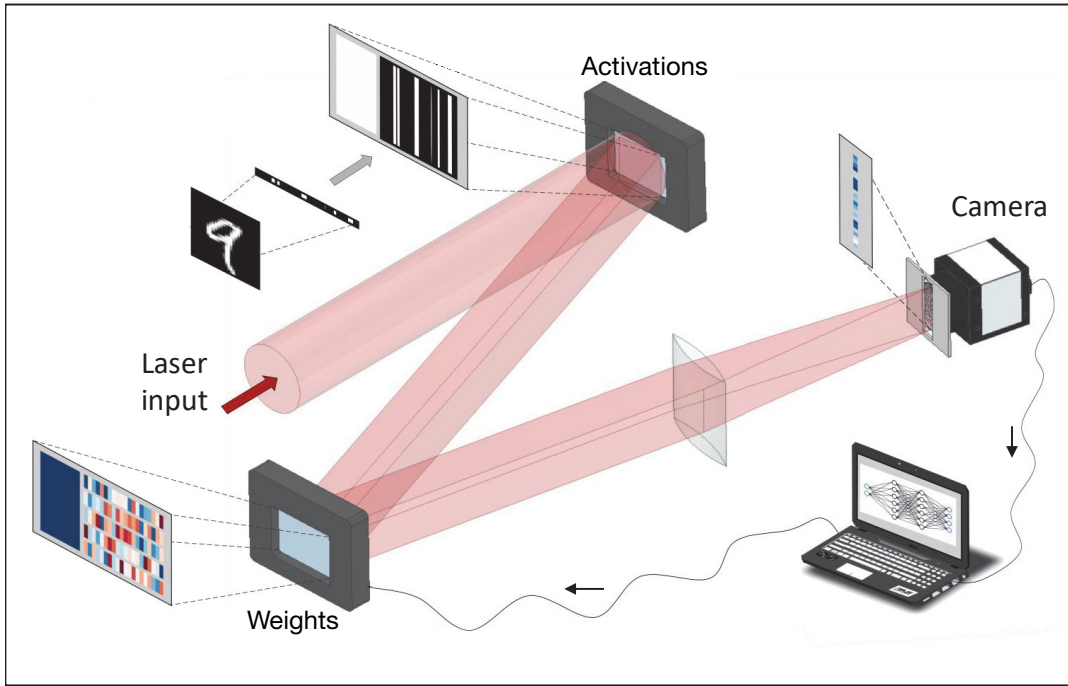


Figure 1: Physical example of an analog matrix–vector multiplication feeding an ADC. The figure, reproduced from Spall, Guo, and Lvovsky [33], shows an optical neural-network front-end: weights and activations are encoded into light fields, the inner product is computed by optical interference, and the analog accumulator output is read by a photodetector before being sampled by an ADC. The same analog-input–ADC-output structure underlies electronic in-memory computing crossbars; the mathematical model in Eqs. (6)–(7) abstracts both.

A small k covers the full worst-case range at the cost of a coarse step Δ , so most of the typical pre-ADC distribution maps into a few central codes. A large k shrinks both the representable range and the step Δ by the same factor, giving a finer lattice on values near zero at the cost of clipping the tails. The relationship is illustrated in Fig. 2.

The default hardware configuration of this thesis fixes $n_t = 256$, $b_x = b_w = 4$, $b_a = 8$, and $k = 16$. All other hardware parameters are held fixed in this default; k itself is varied in a dedicated per-layer k -search experiment in Sec. 5.5, which selects an ADC scaling factor independently for each projection layer in the Unsigned setting.

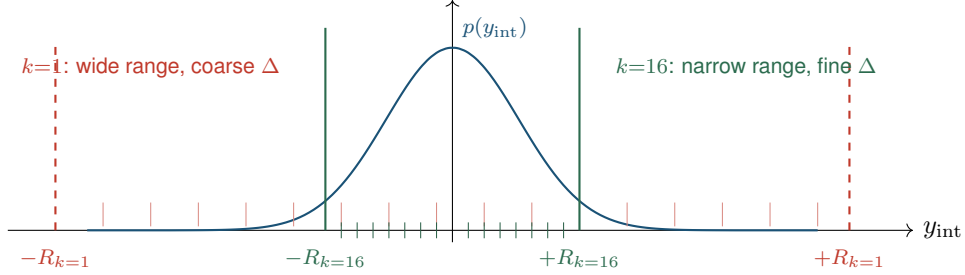


Figure 2: Role of the ADC scaling factor k . The blue curve is the pre-ADC distribution of y_{int} . The red dashed markers show the representable ADC range for $k = 1$: it covers the full worst-case range $\pm n_t q_x q_w$, but the bin width Δ is so wide that most of the distribution maps into a few central codes. The green solid markers show the same ADC for $k = 16$: the representable range shrinks $16\times$ but Δ also shrinks $16\times$, so values near zero are resolved with $16\times$ finer steps while values in the tails are clipped. This thesis uses $k = 16$.

Two failure modes follow directly from this discretisation, both shown in Fig. 3. The dead zone is asymmetric under floor quantization: values in $0 \leq y_{\text{int}} < \Delta$ map to the zero code, whereas small negative values $-\Delta \leq y_{\text{int}} < 0$ map to the -1 code rather than zero. The dead-rate is therefore measured directly as the fraction of post-ADC codes equal to zero (Eq. 36). Clipping occurs when $|y_{\text{int}}| > \Delta \cdot (2^{b_a-1} - 1)$, that is, when the analog accumulation exceeds the representable ADC range.

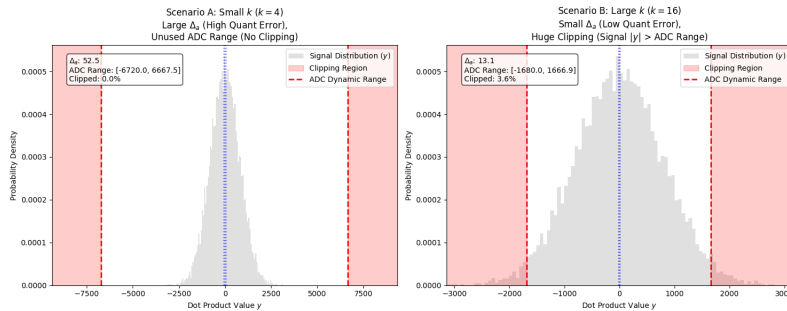


Figure 3: ADC dead-zone illustration. Pre-ADC values in $[0, \Delta)$ floor to the zero code, discarding the analog signal entirely. The two failure modes (dead zone and clipping) are determined by the fixed step Δ and the b_a -bit ADC range.

The ADC quantizer described above is qualitatively different from the quantization stages on the digital side of the pipeline. In purely digital low-bit inference, the dominant errors come from the input and weight quantizers and from approximate integer arithmetic, and

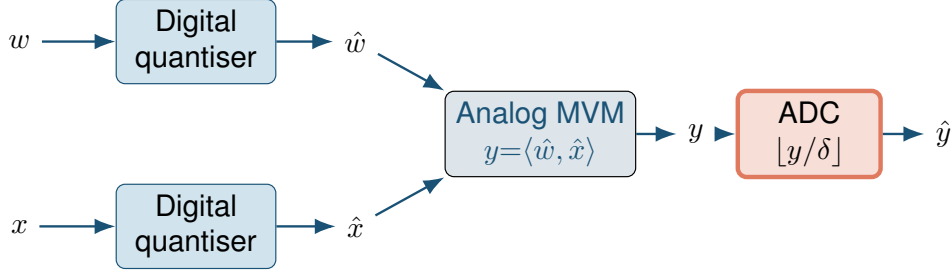


Figure 4: Math model of an analog linear layer. Activations x and weights w are first quantized digitally to $\hat{x}$ and $\hat{w}$. The analog MVM engine then computes the inner product $y = \langle \hat{w}, \hat{x} \rangle$ in continuous-valued form. Finally the ADC maps y to a discrete output $\hat{y}$ via $\lfloor y/\delta \rfloor$, with step size δ fixed by the hardware. The two distinct quantization stages (digital input, analog-to-digital output) are formalised in Eqs. (6)–(7).

all of them can be tightened by better calibration. The ADC step Δ , by contrast, is fixed at manufacturing time and cannot be reduced by retraining; even if Q_x and Q_w are well calibrated, the accumulated output can still be projected onto an insufficiently fine ADC lattice [34, 5]. The main bottleneck may therefore shift from input-side quantization to output quantization, and this distinction is central to the present thesis.

The problem becomes more severe for transformer models. Their hidden states are high-dimensional, anisotropic, and may contain rare channels with extremely large magnitude. These outliers stress the activation quantizer before the analog array and also distort the distribution of y_{int} presented to the ADC. Consequently, the question is how to quantize weights and activations and how to shape internal signals so that the analog accumulation uses the ADC range efficiently [34, 5].

2.2 Quantization of Neural Networks

Large neural networks, and large language models in particular, are bottlenecked by memory bandwidth and the arithmetic throughput of dense linear layers in floating-point precision. Quantization addresses this by representing weights and activations as low-bit integers, trading numerical accuracy for substantial reductions in model size and compute cost. In analog or IMC settings this is more than an optimization: the hardware itself operates at a fixed and often low precision, so the digital side of the pipeline must match it.

Quantization maps continuous tensors to a finite set of discrete values in order to reduce memory, bandwidth, and arithmetic cost [19, 37, 16]. In symmetric uniform quantization, a scalar $x \in \mathbb{R}$ is mapped to

$$q(x) = \text{clip}\left(\left\lfloor \frac{x}{s} \right\rfloor, -q_{\max}, q_{\max}\right), \quad \hat{x} = s q(x), \quad (8)$$

where $s > 0$ is the quantization scale and $q_{\max} = 2^{b-1} - 1$ for a signed b -bit quantizer. Symmetric quantization is widely used for weights and signed activations because it preserves zero exactly and simplifies integer arithmetic.

In asymmetric quantization, one adds a zero-point z :

$$q(x) = \text{clip}\left(\left\lfloor \frac{x}{s} + z \right\rfloor, q_{\min}, q_{\max}\right), \quad \hat{x} = s (q(x) - z). \quad (9)$$

This is useful when the signal distribution is non-negative or strongly shifted, as in some post-activation tensors or hardware-aware activation coding schemes [19, 37].

Quantization also differs in granularity. In per-tensor quantization, one scale is shared by the full tensor. In per-channel quantization, each output channel receives its own scale; this is particularly useful for weights because different rows of W often have very different ranges. For transformer activations, one also encounters per-token and per-group scaling, especially when the activation distribution changes strongly across tokens or heads.

Two training regimes dominate the literature. In post-training quantization (PTQ), a pre-trained floating-point model is quantized after training, usually using a small calibration set and a local reconstruction objective [40, 21, 32, 12, 23, 1]. A generic PTQ objective for layer ℓ can be written as

$$\min_{\theta_\ell^q} \sum_{x \in \mathcal{C}} \left\| f_\ell(x; \theta_\ell) - \hat{f}_\ell(x; \theta_\ell^q) \right\|_2^2, \quad (10)$$

where $\mathcal{C}$ is a calibration set and θ_ℓ^q are the quantized parameters or auxiliary calibration variables. PTQ is attractive because it avoids retraining the full model.

In quantization-aware training (QAT), quantization is inserted into the forward pass during training or finetuning:

$$\min_{\theta} \sum_{(x,y)} \mathcal{L}(y, f_q(x; \theta)), \quad (11)$$

so that the parameters adapt directly to quantization noise [8, 14, 22, 7]. Since rounding is non-differentiable, QAT commonly relies on the straight-through estimator (STE), which uses the quantized function in the forward pass but approximates its gradient in the backward pass by

$$\frac{\partial q(x)}{\partial x} \approx 1. \quad (12)$$

Although heuristic, STE is one of the standard tools behind low-bit QAT and learned PTQ methods [8, 14].

Several refinements are especially relevant for the present work. PACT learns activation clipping thresholds [8]; LSQ learns the step size itself [14]; AdaRound learns non-trivial rounding decisions in PTQ [27]; and QDrop introduces stochastic activation dequantization during calibration to stabilize extremely low-bit PTQ [36]. These techniques all point to the same theme: in low-bit settings, choosing the bitwidth is not enough; one must also decide how the signal is clipped, scaled, rounded, and propagated.

2.3 Challenges of Quantizing Transformer Models

Transformer language models are difficult to quantize. A major reason is the presence of activation outliers. Dettmers et al. introduced LLM.int8() [11], the first work to show that hidden states in transformers contain rare but very large feature values, and that preserving these outlier dimensions matters even when the rest of the model is quantized aggressively. The same phenomenon motivates later methods such as SmoothQuant, Outlier Suppression+, QuaRot, SpinQuant, and FlatQuant [38, 35, 2, 24, 23].

Figure 5 illustrates this on a single decoder block of Llama-3.2-1B: one hidden dimension carries values two orders of magnitude larger than the rest.

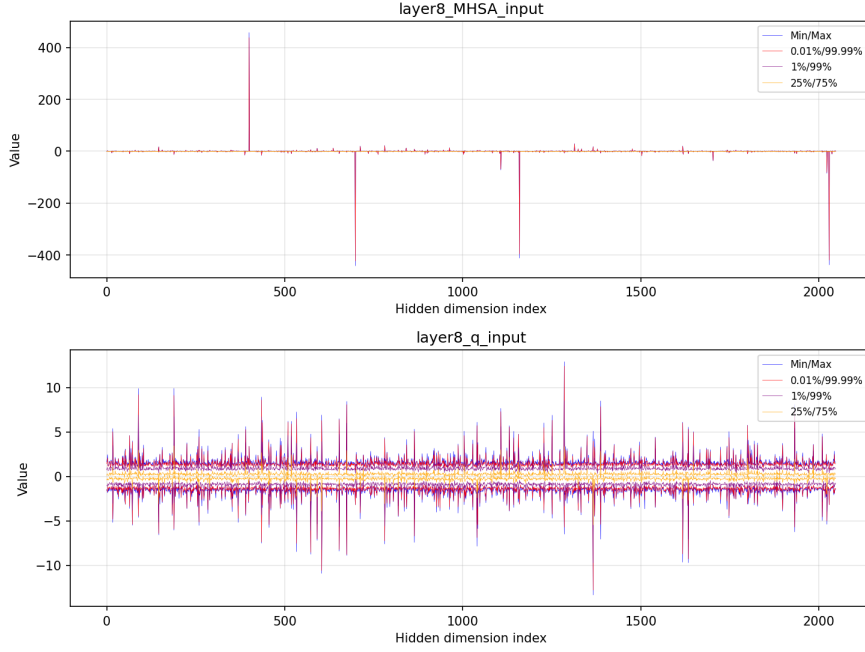


Figure 5: Per-channel activation statistics from a single Llama-3.2-1B decoder block (layer 8), measured over the WikiText-2 calibration set. *Top*: the input to the multi-head self-attention sub-layer (MHSA *input*, i.e. the post-RMSNorm hidden state). A single hidden dimension reaches values on the order of ± 400 , while the bulk of channels stays within ± 10 . *Bottom*: the input to the query projection (*q_input*) shows the same channel structure compressed into a ± 10 range. The colored bands report min/max, 0.01/99.99%, 1/99%, and 25/75% percentiles across the calibration tokens. Under max-based per-token scaling (Eq. (13)), a single such outlier channel dominates the scale and forces typical activations into a narrow range of integer codes, which is the underlying cause of dead-zone behaviour in the downstream ADC.

This is especially problematic under max-based per-token scaling. For a signed activation tensor, a common choice is

$$s_x = \frac{\|x\|_\infty}{q_{\max}}, \quad (13)$$

which ensures that the largest feature maps to the largest available code. If one channel has magnitude M and the remaining channels are of order $\varepsilon \ll M$, then most quantization levels are allocated to that single channel. In low-bit settings, many of the smaller features are then quantized to zero or to a very narrow code band. In an ADC-constrained pipeline, this not only harms the input code density, but also leads to poorly occupied output bins after analog accumulation.

A second challenge is the sensitivity of the residual stream. Residual connections propagate local errors across many layers, so local reconstruction loss is not always predictive of end-to-end perplexity. QuaRot explicitly motivates its method by the need to remove outliers from the residual stream [2], and SpinQuant shows that choosing or learning a suitable rotation can dramatically change the accuracy of low-bit LLM inference [24]. QEP later argues that one of the central failures of standard layer-wise PTQ is the accumulation of quantization errors across layers, and proposes explicit quantization-error propagation to mitigate this effect [1]. CBQ reaches a related conclusion from a cross-block reconstruction perspective [12].

A third challenge is architectural non-uniformity. Not all transformer projections are equally sensitive. The output projections `o_proj` and `down_proj` inject their errors directly into the residual stream, whereas earlier projections such as `q_proj`, `k_proj`, `v_proj`, `up_proj`, and `gate_proj` are followed by additional mixing or nonlinear structure before their effect reaches the next block. This asymmetry is one reason why output-side projections often emerge as the most sensitive modules in low-bit transformer inference.

Finally, preserving hidden-state geometry is critical. A transform that reduces scalar range at the price of severely distorting the hidden-state subspace may still harm model quality. Rotation- and affine-transform methods are therefore attractive because they attempt to make the model more quantizable while preserving the underlying full-precision linear map. This point is especially relevant in the present thesis, where the goal is to shape the signal path so that it tolerates both ordinary low-bit quantization and ADC quantization at the output of each analog MVM, beyond simply reducing local error.

2.4 Existing Methods

Research on low-bit PTQ for LLMs has converged around a small number of recurring ideas described in what follows. Two of these methods, FlatQuant and RAOQ, are central to this thesis and receive extended treatment.

The earliest line of work reshapes the distribution of activations before quantisation. SmoothQuant [38] introduces the diagonal equivalent transformation, which absorbs a per-channel diagonal D into the weights so that the linear map is preserved:

$$y = (xD^{-1})(DW)^\top = xW^\top. \quad (14)$$

With D chosen from activation and weight statistics, the rescaled activation $\tilde{x} = xD^{-1}$ has reduced peak magnitude, so per-token max-based scaling no longer collapses around a single outlier channel. Outlier Suppression+ [35] extends this scheme with a per-channel shift that handles asymmetric outliers; AWQ [21] uses activation statistics to identify and

protect a small set of salient weights; and OmniQuant [32] makes both the diagonal transformation (LET) and a weight-clipping range (LWC) end-to-end learnable, pushing PTQ to very low-bit settings such as W4A4.

These methods reshape the activation distribution but leave the calibration objective itself unchanged. A separate line of work targets that objective directly, treating PTQ as a calibration problem in which the quantisation parameters are fit by minimising reconstruction error against a full-precision reference. ZeroQuant [40] pairs layer-wise distillation with hardware-oriented deployment. CBQ [12] generalises the single-layer objective to cross-block reconstruction, exposing the optimiser to several consecutive transformer blocks at once. QEP [1] feeds quantised hidden states forward during calibration, so that the optimisation target reflects accumulated quantisation error rather than clean teacher activations.

The next line of methods replaces the diagonal scaling of SmoothQuant with a denser linear transformation. QuaRot [2] inserts a fixed Hadamard rotation into the residual stream, removing large-magnitude directions before quantisation. SpinQuant [24] learns this rotation rather than drawing it at random, showing that the choice of rotation substantially affects low-bit accuracy. Both methods act at the level of the residual stream rather than at the level of individual projections.

FlatQuant [23], the calibration backbone of this thesis, sharpens this idea by applying a separate invertible linear transform to each linear projection of the model:

$$y = x W^\top = (x T^{-1}) (T W^\top) = \tilde{x} \tilde{W}^\top. \quad (15)$$

Unlike QuaRot and SpinQuant, the transform T is learned end-to-end through block-level reconstruction, so every projection has its own tailored transform that flattens the local activation distribution. To keep the per-projection parameter count tractable, the $d \times d$ transform is parameterised as a Kronecker product

$$T = T_1 \otimes T_2, \quad T_1 \in \mathbb{R}^{a \times a}, \quad T_2 \in \mathbb{R}^{b \times b}, \quad ab = d, \quad (16)$$

reducing the parameter count from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$. After calibration, T is folded into the surrounding weights, so the inference path is unchanged. At W4A4, FlatQuant set the strongest published PTQ result on Llama-3 at the time of writing.

The methods discussed so far operate entirely in the digital domain. RAOQ [34] is the first to make the analog-to-digital boundary itself an object of optimisation. Unlike the PTQ methods above, RAOQ is a quantisation-aware training (QAT) method: it starts from a pre-trained model and fine-tunes weights and activations end-to-end with the ADC included in the forward pass, targeting the variance of the pre-ADC accumulation $\text{Var}[Y]$, which sets the signal-to-quantisation-noise ratio of the analog readout. Activation shifting (A-shift) re-quantises signed activations as unsigned and shifts them by a constant $C = 2^{b_x-1}$,

$$x_s = \text{clip}\left(\left\lfloor \frac{x - \beta}{s_x} \right\rfloor, 0, 2^{b_x} - 1\right) - C, \quad (17)$$

which concentrates the mass of x_s at large absolute values and increases $\mathbb{E}[X^2]$ by roughly $15\times$ compared with direct signed quantisation; the resulting offset $C \sum_i w_i$ in the IMC computation is precomputed offline and adds no inference overhead. Weight

reshaping (W-reshape) adds a kurtosis penalty to the QAT loss to flatten the weight distribution and increase $\text{Var}[W]$:

$$\kappa = \mathbb{E} \left[\left(\frac{W - \mu_W}{\sigma_W} \right)^4 \right], \quad L_Q = L_c + \lambda_\kappa \sum_{\ell} \kappa_{\ell}, \quad (18)$$

with μ_W, σ_W the mean and standard deviation of the weight tensor and L_c the task loss. Bit augmentation (BitAug) trains the model under multiple ADC bit precisions simultaneously,

$$L_A = L(\theta, b_a) + \lambda_b L(\theta, \tilde{b}_a), \quad (19)$$

with $\tilde{b}_a$ sampled at each iteration from a neighbourhood of the target bit width; the extra gradients smooth the ADC-induced loss surface and reduce the chance of getting stuck in local minima. For models with more than $\sim 200\text{M}$ parameters, RAOQ also fits a LoRA-style adapter (ADC-LoRA) inside the quantised weight, $W + AB$, to reduce the gradient-accumulation cost of BitAug; because AB is summed into the weight before the ADC, this is a pre-ADC LoRA. Analog Foundation Models [5] take a different position within the same QAT-style family: rather than tuning a small set of parameters, the entire LLM is fine-tuned under explicit analog noise constraints, trading training cost for stronger end-to-end accuracy.

When pure PTQ saturates, a natural next step is to freeze the quantised backbone and attach a small trainable correction. LLM-QAT [22] applies quantisation-aware training to extremely low-bit transformers, and EfficientQAT [7] reduces the cost of this idea through block-wise parameter updates and end-to-end training of the quantisation parameters. A related sub-line treats the correction as low-rank: INT2.1 [6], LQER [42], Low-Rank Correction for Quantized LLMs [31], and LR-QAT [4] all show that much of the residual quantisation error can be captured by training a small low-rank addition to each linear layer.

2.5 Research Gap

The literature reviewed above establishes two complementary facts. First, state-of-the-art PTQ methods for LLMs — scaling-, rotation-, and reconstruction-based alike — are highly effective at reducing input-side activation and weight quantization error [38, 21, 32, 2, 24, 23]. Second, IMC-oriented work shows that analog output quantization introduces a separate and often dominant bottleneck that is not naturally captured by ordinary digital PTQ objectives [34, 5].

This creates a specific gap for ADC-constrained LLM inference. Standard PTQ methods typically optimize a local reconstruction objective

$$\mathcal{L}_{\text{PTQ}} = \sum_{x \in \mathcal{C}} \left\| f_{\ell}(x) - \hat{f}_{\ell}(x) \right\|_2^2, \quad (20)$$

possibly with clipping or affine preconditioning. Even when quantization-error propagation is included, the target is usually still a digital hidden-state reconstruction objective [1]. In an IMC accelerator, however, the effective layer map is

$$\hat{f}_{\ell}^{\text{ADC}}(x) = D_{\text{out}} Q_{\text{ADC}}(Q_w(W_{\ell}) Q_x(x)), \quad (21)$$

so the model is constrained not only by low-bit inputs and weights, but also by the ADC quantization of the analog accumulation. Consequently, a method can achieve strong

ordinary PTQ metrics and still perform poorly once the ADC is inserted into the inference loop.

ADC-aware QAT methods such as RAOQ [34] and Analog Foundation Models [5] have shown that this output-side bottleneck can in principle be closed by fine-tuning the entire network end-to-end with the ADC in the loop. The cost of this approach scales poorly with model size: full QAT on a billion-parameter LLM requires hours to days of GPU time per configuration, full-precision gradients through every quantised layer, and additional memory for techniques like BitAug that accumulate gradients across several ADC bit widths. ADC-aware QAT has therefore been demonstrated mostly on convolutional models and on language models in the 100M–1B range, and does not yet scale comfortably to the contemporary LLM regime. The first question raised by this gap is whether transform-based PTQ, which is much cheaper than full QAT, can be extended to be ADC-aware while retaining model quality.

A second gap appears after pure PTQ saturates. Once the best transform-based ADC-aware PTQ solution has been found, the remaining error may no longer be well addressed by additional clipping, scaling, or rotation heuristics. At that point, the relevant question is whether the frozen ADC path can be corrected by a lightweight residual branch rather than by retraining the full model. Although low-rank correction has been studied in digital low-bit settings [42, 31, 4], its interpretation as a post-ADC residual correction on top of a frozen IMC signal path is not standard in the LLM PTQ literature.

The research gap of this thesis is therefore twofold:

1. existing PTQ methods mainly optimize input-side quantization error, while ADC-constrained inference introduces a distinct output quantization bottleneck that must be modeled explicitly;
2. once transform-based PTQ saturates, a lightweight residual correction stage may be required, but it is unclear where such a correction should be inserted and how it should be trained.

The following chapters address these gaps in two stages. First, they extend transform-based PTQ with ADC-aware objectives, error propagation, and bounded equivalent scaling. Second, they investigate low-rank residual correction on top of the frozen quantized ADC path, showing that post-ADC correction can recover a large fraction of the remaining accuracy loss in low-bit LLaMA inference.

3 Problem Statement and Research Questions

3.1 Problem Definition

The core problem studied in this thesis is how to deploy a transformer LLM on an in-memory computing (IMC) inference path in which every large linear projection is constrained both by low-bit weight and activation quantization and by a finite-resolution analog-to-digital converter (ADC) after the matrix–vector multiplication. In a standard digital setting, a linear projection in a transformer block computes

$$y = xW^\top + b, \tag{22}$$

where $x \in \mathbb{R}^{d_{\text{in}}}$ is the input hidden state, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the projection matrix, and b is an optional bias term. In the IMC setting considered here, the same operation is executed

tile by tile in quantized code space.

Let the input dimension be split into tiles of width M . For tile t , the activation and weight tensors are quantized as

$$x_t \approx s_{x,t} \hat{x}_t, \quad W_t \approx \text{diag}(s_{w,t}) \hat{W}_t, \quad (23)$$

where $\hat{x}_t$ and $\hat{W}_t$ are integer activation and weight codes, $s_{x,t}$ is the activation scale, and $s_{w,t}$ is the vector of per-output-channel weight scales. The analog array accumulates the integer dot product

$$z_t = \hat{x}_t \hat{W}_t^\top. \quad (24)$$

Unlike conventional digital quantization, this accumulated value is not immediately available at full integer precision. It must first be discretized by the ADC:

$$Q_{\text{ADC}}(z_t; \Delta, n_a, p_a) = \Delta \cdot \text{clip} \left(\left\lfloor \frac{z_t}{\Delta} \right\rfloor, n_a, p_a \right), \quad (25)$$

where Δ is the ADC step size and (n_a, p_a) are the lower and upper ADC code bounds. The dequantized output of the tiled projection is then

$$\hat{y} = b + \sum_t s_{x,t} (s_{w,t} \odot Q_{\text{ADC}}(z_t; \Delta, n_a, p_a)), \quad (26)$$

where $\odot$ denotes element-wise multiplication.

The ADC step size is a hardware-defined quantity. For the Signed readout used in the main experiments, it is given by

$$\Delta = \frac{2M q_x q_w}{2^{b_a} k}, \quad (27)$$

where b_a is the ADC bit-width, k is the ADC scaling factor, and q_x, q_w are the maximum activation and weight code levels determined by the activation and weight precisions b_x and b_w . For the Unsigned shift-subtract readout, the corresponding unsigned step size is

$$\Delta_u = \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{(2^{b_a} - 1) k}. \quad (28)$$

In both cases, the essential property is the same: the output lattice is fixed by hardware parameters and cannot be made finer by ordinary post-training calibration.

This makes ADC-constrained inference fundamentally different from standard digital post-training quantization (PTQ). A PTQ method may successfully reduce the error in $\hat{x}_t$ and $\hat{W}_t$, while the accumulated output z_t may still be mapped to a coarse ADC lattice. If $0 \leq z_t < \Delta$, the output maps to the zero ADC code under floor quantization; small negative values instead map to the -1 code, so the dead-zone rate is measured directly as the fraction of post-ADC codes equal to zero. If z_t exceeds the ADC range, the output is clipped. Therefore, the objective is to quantize weights and activations accurately and to shape the internal integer accumulation so that the ADC range is used effectively.

The software-side degrees of freedom considered in this thesis are transform-based PTQ parameters and, in the second stage, a lightweight residual adaptation branch. Following the FlatQuant idea, an invertible transform T_ℓ can be inserted into a linear projection without changing the full-precision computation:

$$x W_\ell^\top = (x T_\ell^{-1})(T_\ell W_\ell^\top). \quad (29)$$

However, after quantization and ADC discretization, this equivalence no longer holds exactly. The practical calibration problem is therefore to choose transforms that minimize the mismatch between the full-precision block output and the ADC-constrained quantized block output:

$$\min_{\phi} \mathbb{E}_{x \in \mathcal{C}} \left[\left\| B_{\ell}(x) - \hat{B}_{\ell}^{\text{ADC}}(x; \phi, \mathcal{H}) \right\|_2^2 \right], \quad (30)$$

where B_{ℓ} is the full-precision transformer block, $\hat{B}_{\ell}^{\text{ADC}}$ is its quantized ADC-constrained counterpart, ϕ denotes the PTQ calibration parameters, $\mathcal{C}$ is the calibration set, and

$$\mathcal{H} = (b_x, b_w, b_a, M, k, \text{ADC mode}) \quad (31)$$

is the fixed hardware configuration.

At the model level, the goal is to obtain an ADC-constrained model $\hat{F}^{\text{ADC}}$ whose quality remains close to the full-precision model F and to the corresponding digital quantized model with the ADC disabled:

$$\min_{\phi, \psi} \text{PPL} \left(\hat{F}_{\phi, \psi}^{\text{ADC}}; \mathcal{D}_{\text{eval}} \right), \quad \text{subject to fixed } \mathcal{H} \text{ and } |\psi| \ll |\theta|. \quad (32)$$

Here $\psi = 0$ corresponds to pure PTQ, while ψ denotes the optional lightweight residual correction parameters used in the adaptation stage. The central question is whether ADC-aware PTQ alone is sufficient, or whether the fixed ADC output lattice creates a residual error that must be corrected by a small trainable module.

3.2 Research Questions

This thesis is guided by the following research questions.

RQ1. How much quality degradation is caused specifically by ADC output quantization in LLMs?

This question separates ordinary digital quantization error from the additional degradation introduced by the ADC. The comparison is made between full-precision inference, quantized inference with the ADC disabled, and quantized inference with the full ADC path enabled. The goal is to determine whether the ADC is a minor implementation detail or a dominant source of error under low-bit IMC inference.

RQ2. To what extent can FlatQuant-style post-training transformations mitigate ADC-induced degradation?

This question studies whether learned equivalent transformations can reshape activation and weight distributions so that the integer accumulation uses the ADC range more effectively. In particular, the thesis evaluates block-wise calibration, propagated quantized inputs, mixed full-precision/ADC objectives, trainable diagonal scaling, and staged training.

RQ3. Is reconstruction-only PTQ sufficient under ADC constraints, or are explicitly ADC-aware objectives required?

Standard PTQ methods usually minimize reconstruction error under digital quantization assumptions. This thesis investigates whether such objectives remain predictive once the ADC is inserted into the inference path. The question is answered by comparing clean-input reconstruction, ADC-propagated reconstruction, and objectives that explicitly expose the calibration process to ADC noise.

RQ4. Which layers and projection types are most sensitive to ADC quantization?

LLaMA decoder blocks contain several linear projections: q , k , v , o , gate, up, and down. These projections do not contribute equally to end-to-end degradation. This question identifies whether ADC sensitivity is concentrated in particular projections, such as output-side attention and MLP projections, or distributed uniformly across the architecture.

RQ5. Under what conditions does ADC-constrained inference require lightweight model adaptation beyond PTQ?

If transform-based PTQ reaches a quality plateau, the remaining error may be caused by the finite ADC output resolution rather than by poorly calibrated weights or activations. This question evaluates whether a small post-ADC residual correction branch can close the remaining quality gap while keeping the quantized ADC backbone frozen.

Together, these questions connect three levels of analysis: the hardware-level ADC discretization, the layer-level behavior of quantized transformer projections, and the model-level language modeling quality.

3.3 Scope and Assumptions

The thesis focuses on a decoder-only transformer LLM, with Llama-3.2-1B used as the main experimental target. This model is large enough to exhibit the activation outlier and residual-stream sensitivity typical of modern LLMs, while still being small enough to support extensive ablation studies over quantization precision, ADC settings, calibration strategies, and residual adaptation variants.

The hardware behavior is studied through simulation rather than through measurements on a fabricated IMC chip. The simulator models tiled integer matrix–vector multiplication, per-token activation quantization, per-channel weight quantization, ADC floor quantization, ADC clipping, and dequantization. Two readout settings are considered: **Signed** and **Unsigned**. Signed denotes a signed-accumulation readout with a symmetric ADC code range. Unsigned denotes a single-read unsigned shift-subtract readout, where signed INT4 codes are shifted to non-negative codes before the MVM and the zero-point terms are subtracted digitally after ADC readout. The hardware parameters M , b_x , b_w , b_a , and k are treated as fixed during each experiment unless an explicit per-layer k -search experiment is performed.

Several hardware effects are outside the scope of this work. The simulator does not model device-to-device variation, thermal noise, conductance drift, IR drop, nonlinear crossbar behavior, write noise, peripheral circuit latency, or full chip-level energy consumption. The purpose of the study is therefore not to claim final hardware efficiency for a specific IMC implementation, but to isolate the algorithmic consequences of ADC output discretization.

The quantization study is restricted to the main linear projections inside the transformer decoder blocks. Other components, such as layer normalization, nonlinearities, softmax, rotary embeddings, and token embeddings, are not the primary target of the ADC simulation. This restriction reflects the fact that large linear projections dominate transformer inference cost and are the most natural mapping target for IMC arrays.

The main methodological focus is post-training quantization. The thesis does not retrain the full model and does not modify the original pretraining objective. When adaptation is considered, it is limited to a small residual low-rank branch trained on top of a frozen quantized ADC path. This allows the thesis to distinguish between what can be achieved by calibration alone and what requires additional trainable parameters.

The calibration data is finite and substantially smaller than the original pretraining data. Therefore, the reported results should be interpreted as evidence about practical calibration and adaptation under limited data, not as an upper bound on what could be achieved with large-scale quantization-aware training.

3.4 Evaluation Criteria

The primary model-level metric is perplexity. For each configuration, the thesis compares

$$\text{PPL}_{\text{FP}}, \quad \text{PPL}_{\text{no ADC}}, \quad \text{PPL}_{\text{ADC}}, \quad (33)$$

where PPL_{FP} is the full-precision baseline, $\text{PPL}_{\text{no ADC}}$ measures quantized inference with the ADC disabled, and PPL_{ADC} measures the full ADC-constrained inference path. The difference between no-ADC and ADC perplexity isolates the degradation introduced by output quantization:

$$\Delta_{\text{ADC}} = \text{PPL}_{\text{ADC}} - \text{PPL}_{\text{no ADC}}, \quad (34)$$

while the ratio

$$\rho_{\text{ADC}} = \frac{\text{PPL}_{\text{ADC}}}{\text{PPL}_{\text{no ADC}}} \quad (35)$$

measures the relative severity of the ADC bottleneck. A successful ADC-aware method should reduce both PPL_{ADC} and the no-ADC-to-ADC gap without severely degrading no-ADC quality.

In addition to perplexity, the thesis uses internal ADC-level metrics to diagnose why a configuration succeeds or fails. The dead-zone rate is defined as the fraction of pre-ADC accumulated values whose post-ADC code is exactly zero,

$$r_{\text{dead}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\lfloor z_i/\Delta \rfloor = 0], \quad (36)$$

where z_i denotes a pre-ADC accumulated value. This is the floor-consistent definition: $\lfloor z_i/\Delta \rfloor = 0$ holds for $z_i \in [0, \Delta)$ in both the Signed and Unsigned readouts. A high dead-zone rate indicates that many analog outputs are too small to register on the ADC lattice. The clipping rate is defined as

$$r_{\text{clip}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[c_i = n_a \text{ or } c_i = p_a], \quad (37)$$

where c_i is the ADC code after discretization and clipping. This metric detects whether the integer accumulation systematically exceeds the representable ADC range.

The thesis also evaluates ADC bin utilization:

$$u_{\text{bin}} = \frac{|\{c_i : i = 1, \dots, N\}|}{2^{b_a}}, \quad (38)$$

which measures how much of the available ADC codebook is actually used. Low bin utilization indicates that the ADC behaves like a much lower-precision device than its nominal bit-width suggests.

Layer-level reconstruction quality is measured by the relative error

$$e_\ell = \frac{\|\hat{y}_\ell - y_\ell^*\|_2^2}{\|y_\ell^*\|_2^2}, \quad (39)$$

where y_ℓ^* is the full-precision reference output and $\hat{y}_\ell$ is the quantized ADC output. This metric is used to identify local failures even when the final perplexity alone does not reveal their source.

Architectural sensitivity is studied at the projection level. For a given projection p , the per-projection ADC sensitivity is defined as

$$\Delta_{\text{ADC}}^p = \text{PPL}_{\text{ADC}} - \text{PPL}_{\text{ADC} \setminus p}, \quad (40)$$

where $\text{PPL}_{\text{ADC} \setminus p}$ is the perplexity when the ADC is switched off on p alone, while digital quantisation and the ADC on every other projection remain active. Summing Δ_{ADC}^p over all projections approximately recovers the global Δ_{ADC} ; the individual values reveal whether the ADC bottleneck is concentrated in a few projections or distributed uniformly across the architecture.

For residual adaptation experiments, three metrics are tracked together. The closing fraction

$$\eta = \frac{\text{PPL}_{\text{ADC}}^{\text{PTQ}} - \text{PPL}_{\text{ADC}}^{\text{PTQ+LoRA}}}{\text{PPL}_{\text{ADC}}^{\text{PTQ}} - \text{PPL}_{\text{FP}}} \quad (41)$$

measures how much of the remaining PTQ-to-FP gap is closed by the adaptation. The parameter overhead

$$\rho_{\text{params}} = \frac{\sum_\ell (|A_\ell| + |B_\ell|)}{\sum_\ell |W_\ell|} \quad (42)$$

measures the size of the added correction relative to the base model, where A_ℓ, B_ℓ are the low-rank factors at layer ℓ . Out-of-domain generalisation is measured by the C4 ADC perplexity of the same LoRA checkpoint without re-calibration. A successful method achieves η close to one with small ρ_{params} and a small C4 ADC perplexity gap.

4 Proposed Method

4.1 Post-Training Quantization

4.1.1 Baseline: FlatQuant for LLaMA

As discussed in Sec. 2.4, FlatQuant [23] inserts a learnable invertible transform $T = T_1 \otimes T_2$ into each linear projection and folds it into the surrounding weights after calibration, so the deployed inference cost is unchanged. The Kronecker factor sizes are chosen automatically to factor each projection’s input dimension d_{in} into a pair of integers (p, q) with $p \cdot q = d_{\text{in}}$ and $p \approx q$, so that $T_1 \in \mathbb{R}^{p \times p}$, $T_2 \in \mathbb{R}^{q \times q}$, and the parameter count grows as $\mathcal{O}(p^2 + q^2) = \mathcal{O}(d_{\text{in}})$ rather than $\mathcal{O}(d_{\text{in}}^2)$.

For Llama-3.2-1B (residual width $d = 2048$, FFN intermediate width $d_{\text{ffn}} = 8192$) the per-projection factorisations are therefore not uniform. The six projections whose input is the

residual stream — q, k, v , and o in the grouped-query attention sub-layer, and `gate` and `up` in the SwiGLU feed-forward network — have $d_{\text{in}} = 2048$ and use 32×64 Kronecker factors. The remaining projection, `down`, takes the wider FFN activation as input ($d_{\text{in}} = 8192$) and uses 64×128 factors accordingly. All seven projections in each of the 16 decoder blocks are wrapped, for a total of $7 \times 16 = 112$ transforms across the model.

Calibration uses a teacher–student objective computed at the transformer-block level. For each block B , a frozen full-precision teacher produces a reference output $y^* = B_{\text{FP}}(x^*)$, and the quantised student $\hat{B}_T$ is optimised over the Kronecker factors of all seven projections inside the block, with weights kept at their quantised values. Gradients therefore propagate through the full block sequence $q/k/v \rightarrow \text{attn} \rightarrow o \rightarrow \text{gate/up} \rightarrow \text{down}$, so the transforms of earlier projections are aware of how their error reaches later ones. The ADC quantiser of Eq. (6) is included in the student’s forward pass, so calibration sees the same floor and clipping non-linearities as deployment. The non-differentiable round and floor operations of the weight, activation, and ADC quantisers are handled by the straight-through estimator (STE).

4.1.2 Cross-Block Propagation

The block-level objective of the baseline above uses the clean teacher input x^* at every block,

$$\mathcal{L}_{\text{FP}} = \|\hat{B}(x^*) - B_{\text{FP}}(x^*)\|_2^2, \quad (43)$$

whereas at inference each block receives the ADC-quantised output of the preceding one. To close this train–deploy mismatch, blocks are calibrated sequentially from layer 0 to layer $N-1$. When training layer ℓ , its input is the ADC-quantised output produced by the already-trained layer $\ell-1$ rather than a clean FP activation. The propagated loss

$$\mathcal{L}_{\text{ADC}} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2, \quad (44)$$

where $\hat{x}$ is the ADC-quantised input from the previous block, matches the training distribution to the deployment distribution.

Sequential block-by-block calibration is slower than calibrating all blocks in parallel, because each block depends on the trained transforms of the preceding ones. An alternative route is taken by AIMC work that approximates the ADC as additive noise on top of the analog accumulation [39, 20, 5]: with such a noise model the input to each layer can be sampled directly, and every layer can be calibrated in parallel without waiting for the preceding ones. This trades the exact floor-and-clip ADC model used here for a smoother stochastic surrogate. The present thesis keeps the exact ADC model and accepts the sequential cost; the noise-based parallelisation is out of scope but is in principle compatible with the same set of ADC-aware extensions.

4.1.3 α -Mixing of the FP and ADC Objectives

Training only on $\mathcal{L}_{\text{ADC}}$ makes the gradients unstable: the straight-through estimator passes through several floor operations, and the Kronecker rotations start to overfit to the ADC noise rather than to the underlying signal. The opposite also fails: training only on $\mathcal{L}_{\text{FP}}$ keeps the gradients clean, but the block never learns to handle the ADC distortion it will see at inference. A mixed objective addresses both failure modes by interpolating the two losses with a scalar $\alpha \in [0, 1]$:

$$\mathcal{L}(\alpha) = (1 - \alpha) \mathcal{L}_{\text{FP}} + \alpha \mathcal{L}_{\text{ADC}}. \quad (45)$$

Intuitively, $\mathcal{L}_{\text{FP}}$ supplies a stable learning signal that anchors the transform to the FP teacher, while $\mathcal{L}_{\text{ADC}}$ teaches the block to tolerate the discretisation introduced by the ADC. The value $\alpha = 0$ recovers clean calibration and $\alpha = 1$ is full propagation.

In the INT8 setting, full propagation alone drops ADC perplexity from 28.86 to 14.55, and adding a small bin-center regulariser further reduces it to 14.41, so an α -sweep is not needed because the noise from full propagation is already manageable. In the INT4 setting full propagation is too noisy: the α -sweep in Section 5 shows that $\alpha = 0.5$ gives the best trade-off, with smaller α under-weighting the ADC regime and larger α degrading transform quality through gradient noise from the heavier quantisation.

4.1.4 Bounded FlatQuant Scaling and Placement

The bound established below applies to the orthogonal part of the transform and explains why rotation-like transformations alone have limited ability to suppress persistent per-channel outliers. FlatQuant already includes a learnable per-channel scaling vector as part of its affine pre-quantisation transformation [23], and the implementation also parameterises each Kronecker factor as $U \text{diag}(s) V^\top$ (see Sec. A.2), so the deployed transform is not purely rotational even before the external scaling is added. In this thesis we use the FlatQuant scaling component in the ADC-aware setting and focus on three aspects that are not addressed by the original digital PTQ formulation: a bounded form for the external per-channel scaling, its placement across attention and MLP sub-layers, and its staged calibration under ADC-propagated inputs.

Lemma 4.1 (Orthogonal rotation-only outlier suppression bound).

This lemma analyses the strict rotation-only subfamily of the transform, in which the Kronecker factors are orthogonal and any internal singular values or external diagonal scalings are fixed to one. Concretely, let $P = L \otimes R$ with $L^\top L = I$, $R^\top R = I$ be an orthogonal Kronecker transform on $\mathbb{R}^d$ ($d = a \cdot b$, $L \in \mathbb{R}^{a \times a}$, $R \in \mathbb{R}^{b \times b}$), and assume no additional diagonal or singular-value scaling is applied. For any $x \in \mathbb{R}^d$ define the effective participation ratio

$$m_{\text{eff}}(x) = \frac{\|x\|_2^2}{\|x\|_\infty^2} \in [1, d].$$

Then for every such orthogonal Kronecker transform P ,

$$\|xP\|_\infty \geq \frac{\|x\|_2}{\sqrt{d}},$$

and consequently the maximum achievable suppression of the peak activation over all such P satisfies

$$\frac{\|x\|_\infty}{\min_P \|xP\|_\infty} \leq \sqrt{\frac{d}{m_{\text{eff}}(x)}}.$$

Proof. Since P is orthogonal,

$$\|xP\|_2 = \|x\|_2.$$

For any vector $y \in \mathbb{R}^d$ the standard ℓ^∞ – ℓ^2 inequality gives

$$\|y\|_\infty \geq \frac{\|y\|_2}{\sqrt{d}}.$$

Applying this to $y = xP$ yields

$$\|xP\|_\infty \geq \frac{\|xP\|_2}{\sqrt{d}} = \frac{\|x\|_2}{\sqrt{d}},$$

which is the first claim. The suppression ratio bound then follows from the definition

$$\|x\|_\infty = \frac{\|x\|_2}{\sqrt{m_{\text{eff}}(x)}}$$

combined with the lower bound on $\|xP\|_\infty$ above:

$$\frac{\|x\|_\infty}{\|xP\|_\infty} \leq \frac{\|x\|_2 / \sqrt{m_{\text{eff}}(x)}}{\|x\|_2 / \sqrt{d}} = \sqrt{\frac{d}{m_{\text{eff}}(x)}}.$$

□

The bound above is not a claim about the full implemented FlatQuant parameterisation. In the implementation each Kronecker factor is represented as $U_i \text{diag}(s_i) V_i^\top$ (Sec. A.2), and the external per-channel scaling vector D is exposed through the `diag_scale` parameter. Once either the internal singular values s_i or the external diagonal D are learned, the transform is no longer orthogonal and Lemma 4.1 does not directly apply. The internal singular values s_L, s_R do introduce anisotropy, but it is Kronecker-separable: the effective singular values of $T_L \otimes T_R$ factor as $s_i^{(L)} s_j^{(R)}$, giving only $a + b$ degrees of freedom, whereas the external per-channel diagonal D gives $d = ab$ independent channel scales. The lemma is therefore used as a diagnostic for the strict rotation-only subcase; the experiments then study how the FlatQuant scaling component behaves under ADC-aware calibration.

Corollary 4.2 (Kronecker capacity).

A full orthogonal matrix on $\mathbb{R}^d$ has $d(d-1)/2$ free parameters. The Kronecker family $L \otimes R$ has only $a(a-1)/2 + b(b-1)/2$ free parameters; for $d = 2048 = 32 \times 64$ this is 2512 vs. $\approx 2.1 \times 10^6$. Consequently, the Kronecker family is a strict low-dimensional subset of all orthogonal transforms and may not attain the bound in Lemma 4.1; the true achievable suppression ratio can be strictly smaller.

In the present setting, per-token activation scaling sets $s_x = \|x\|_\infty / q_{\text{max}}$. If a small number of channels persistently dominate (i.e. $m_{\text{eff}} \ll d$), Lemma 4.1 gives a ceiling on how much any purely rotational transform can help. For $d = 2048$ and 8 dominant channels ($m_{\text{eff}} \approx 8$), the maximum suppression factor is $\sqrt{2048/8} = 16$; when the required suppression exceeds this, the orthogonal component of the transform alone cannot escape the bound. The learnable singular values inside the Kronecker factors relax this constraint somewhat but remain structurally coupled to the Kronecker product, so they do not provide direct per-channel control either. The activation distributions discussed in Section 2.3 and shown in Figure 5 are exactly this regime: a single hidden dimension carries values two orders of magnitude above the typical scale, so $m_{\text{eff}} \ll d$. This is what the explicit per-channel diagonal is added for.

The mechanism is geometric. A Kronecker rotation reshapes the covariance of an activation cloud but does not adjust per-channel scales, and the residual outlier channels that remain after rotation (a few channels with magnitudes 5–10 $\times$ the median) force the

per-token scale factor to waste range on those channels and under-represent the rest. This worsens the ADC dead-zone rate because y_{int} per output channel depends on how uniformly codes are distributed across the output range.

We therefore retain the FlatQuant learnable scaling component and write the external per-channel part explicitly as $D = \text{diag}(d_1, \dots, d_c)$, attached to the rotation as $\tilde{T} = T \cdot D$ and absorbed by an equal-and-opposite rescaling of the weights:

$$y = \underbrace{(x D^{-1} T^{-1})}_{\tilde{x}} \underbrace{(T D W^\top)}_{\tilde{W}^\top}.$$

This diagonal is not claimed as a new equivalent transformation; it is the FlatQuant scaling component reused in the ADC-aware setting. Our contribution in this section is its bounded form, sub-system placement, and staged training schedule (Sec. 5.4.4). The bound is enforced as a projection step in the optimiser, $d_i \in [10^{-4}, 10]$, which prevents degenerate scaling without altering the calibration objective. D is trained jointly with T during block-level calibration and folded into $\tilde{W}$ at deployment, so the inference cost is unchanged. Unlike the strict rotation-only subfamily analysed in Lemma 4.1, D changes per-channel magnitudes directly and therefore escapes the ℓ^2 -energy invariance assumed there: picking $d_j > 1$ amplifies row j of $\tilde{W}$ (where it is absorbed cleanly by per-channel weight scaling) and equivalently shrinks channel j of the rotated activation $\tilde{x}$ by a factor $1/d_j$. For an outlier activation channel this is exactly the per-channel rebalancing that per-token activation scaling cannot perform.

Separate diagonals are placed before the attention sub-layer (one shared by $q/k/v$, plus a second instance before o) and before the FFN sub-layer (one shared by gate and up , plus a second before down). We refer to these two as the *attention diagonal* and the *MLP diagonal*. An ablation (Section 5) shows that both placements are necessary: the MLP diagonal drives the no-ADC perplexity improvement while the attention diagonal closes the ADC gap; either alone leaves significant perplexity on the table.

In the implementation, the external per-channel diagonal D studied in this section corresponds to the optional `diag_scale` parameter (gated by the `add_diag` flag). The two further parameters `diag_left` and `diag_right` that appear in Listing 2 are internal singular values of the individual Kronecker factors T_1 and T_2 ; they are part of the SVD-style Kronecker parameterisation and are *not* counted as the per-channel diagonal component studied in the placement and staging ablations of Section 5.4.4.

4.1.5 Staged Training

Training all Kronecker factors and diagonals jointly from scratch with $\alpha = 0.5$ is unstable: attention gradients dominate early optimization before MLP transforms have converged, yielding a poor initialization for both sub-systems.

Calibration is split into two stages. In both stages the Kronecker factors of all seven projections ($q, k, v, o, \text{gate}, \text{up}, \text{down}$) remain trainable; the staging acts on the per-channel diagonals $D^{(\text{m})}$ and $D^{(\text{a})}$ and on the learning rate. Stage A (30 epochs, learning rate η , $\alpha = 0$) enables only the MLP diagonals $D^{(\text{m})}$ and freezes the attention diagonals $D^{(\text{a})}$; the Kronecker factors are optimised throughout under the clean-input reconstruction loss, while the MLP scaling converges in a stable, noise-free regime. Stage B (10 epochs, learning rate 0.1η , $\alpha = 0.5$) freezes $D^{(\text{m})}$ at its Stage-A value and enables $D^{(\text{a})}$; the Kronecker factors continue to be optimised but with a $10\times$ lower learning rate, so the

MLP basis is effectively held in place while the attention diagonal is fitted under ADC-propagated inputs. The warm MLP start prevents gradient interference between the two sub-systems.

Among configurations that reach the pure-PTQ INT4 plateau, this procedure attains the best no-ADC perplexity, indicating a better-conditioned underlying transform.

4.2 Quantization-Aware Adaptation via Post-ADC Residual LoRA

PTQ methods reach a floor in the high-20s of ADC perplexity for INT4 (around 26–28) regardless of calibration strategy. The limiting factor is ADC resolution: with $\delta \approx 6.12$ each output channel has at most ~ 30 usable discrete levels, introducing irreducible rounding error that no linear transform can eliminate. PTQ is complemented by a lightweight adaptation step that learns to correct ADC distortion after it has occurred, in the digital domain, without modifying the frozen quantized model.

4.2.1 Architecture

The PTQ checkpoint is frozen entirely. For each of the seven linear projections in every decoder block, a parallel low-rank branch is added that reads the full-precision input x_{fp} and adds its output to the ADC result:

$$y = \underbrace{\text{ADC}_{\text{frozen}}(\hat{x})}_{\text{quantized path}} + \underbrace{\frac{\alpha_r}{r} B(A(x_{\text{fp}}))}_{\text{residual correction, rank } r},$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are LoRA [18] factors stored and computed in `float32`, and α_r/r is the standard LoRA scaling. The default configuration uses rank $r = 4$ on all seven projections of every decoder block, with $\alpha_r = 8$ (scaling factor 2.0). All base model parameters (weights, Kronecker factors, diagonals, and ADC step sizes) remain frozen throughout. The added rank-4 adapters contribute roughly 0.22% extra parameters relative to the base model; the full overhead breakdown across rank and target choices is reported alongside the empirical sweep in Sec. 5.

4.2.2 Training Objective

The adapters are trained for 5 epochs with learning rate 10^{-4} using AdamW on the same 1024-sample WikiText-2 calibration set used by the staged PTQ checkpoint of Sec. 5.4.4. The loss combines cross-entropy on the next-token prediction with a knowledge-distillation term toward the FP teacher:

$$\mathcal{L}_{\text{LoRA}} = \text{CE}(\hat{p}, y) + \lambda \text{KL}(\hat{p}_T \parallel p_{\text{FP}, T}),$$

with $\lambda = 0.5$ and temperature $T = 2.0$ on both teacher and student logits. The FP teacher forward pass is computed once per batch with no gradient. Ablations (Section 5) show that the KL term improves ADC perplexity over CE alone, that rank $r = 4$ saturates the gain, and that covering all 16 decoder blocks is necessary because ADC distortion accumulates across depth.

4.2.3 Why Post-ADC Placement

Placing the LoRA correction before the ADC, modifying the pre-ADC integer signal, fails to converge. Any correction $\Delta = B A x$ is itself quantized by the fixed step δ before reaching the output, so it cannot express sub- δ adjustments. Moreover, gradients through the floor operator are too noisy for stable training from a cold start, and the coupled interaction with the hard ADC clamp causes the training run to diverge.

Post-ADC placement avoids both problems: the correction is added in the digital domain after the ADC read, at full floating-point precision, with clean gradients from the CE+KL loss. As a result, ADC perplexity drops below the PTQ plateau on WikiText-2 and surpasses the deployed INT8 + ADC PTQ checkpoint used in Table 3, not the ideal no-ADC INT8 digital baseline. The same configuration also improves C4 ADC perplexity, confirming that the correction generalises beyond the calibration domain.

5 Experiments and Results

5.1 Model, Data, and Hardware Setup

Model. All experiments use Llama-3.2-1B [13] in `bf16`. The model has 16 decoder layers, hidden dimension $d = 2048$, 32 attention heads with grouped-query attention, and a SwiGLU FFN with intermediate dimension 8192. This model is chosen because it admits extensive calibration and ablation experiments within a single-GPU budget while exhibiting the architectural features (grouped-query attention, SwiGLU FFN, RMSNorm) shared with larger decoder-only transformer LLMs.

Calibration data. FlatQuant transforms and LoRA adapters are calibrated on WikiText-2 training text [25]. Unless stated otherwise, 128 randomly sampled sequences of 2048 tokens each are used. Selected experiments use 512 or 1024 samples to study the effect of calibration set size; the INT4 series demonstrates that sample count is a dominant lever for this bit-width.

Evaluation. Perplexity is measured on the WikiText-2 test set using a sliding window of context length 2048 and stride 1024. Selected configurations are also evaluated on C4 [29] under the same protocol to verify out-of-domain generalization.

ADC hardware parameters. Table 1 lists the fixed hardware constants used throughout. All values are chosen to match a plausible near-term AIMC chip target. The ADC step size δ is fully determined by these constants via $\delta = 2Mq_xq_w/(2^{b_a}k)$, giving $\delta \approx 2016$ for INT8 and $\delta \approx 6.12$ for INT4.

FlatQuant calibration settings. Transforms are trained for 30 epochs with learning rate 5×10^{-3} using the AdamW optimizer. Stage B of staged training uses 10 epochs at 5×10^{-4} . Static activation scales for Step 2 of the FlatQuant pipeline use the 99.9th-percentile clipping strategy.

Table 1: Fixed ADC hardware parameters.

Parameter	Symbol	Value
Tile size (crossbar columns)	M	256
ADC bit-width	b_a	8
ADC scaling factor	k	16
INT8 weight range	q_w	127
INT8 activation range	q_x	127
INT4 weight range	q_w	7
INT4 activation range	q_x	7
ADC step (INT8)	δ	≈ 2016
ADC step (INT4)	δ	≈ 6.12

Compute infrastructure. All experiments in this thesis (FlatQuant calibration, post-ADC LoRA training, and perplexity evaluation) are run on a single NVIDIA A100 (80 GB) GPU.

5.2 Evaluation Protocol

All metrics used in the experiments are defined formally in Sec. 3.4.

All perplexity numbers reported in this thesis follow the same protocol: sliding-window evaluation on the test split with context length 2048 and stride 1024. The model is loaded in `bf16`; this is referred to as the full-precision baseline. Under this protocol Llama-3.2-1B gives WikiText-2 PPL 8.68 and C4 PPL 13.13. All subsequent tables compare against these numbers.

In addition to perplexity, the main results table reports accuracy on eight downstream benchmarks from the `lm-evaluation-harness` [15]: HellaSwag [41], a multiple-choice commonsense sentence-completion task; MMLU [17], a four-way multiple-choice benchmark covering 57 academic and professional subjects; WinoGrande [30], a commonsense coreference resolution task; ARC-Easy and ARC-Challenge [10], two difficulty levels of grade-school science question answering; PIQA [3], physical commonsense reasoning; OpenBookQA [26], elementary science question answering with open-book style facts; and BoolQ [9], yes/no reading comprehension. Table 2 summarises the configuration used for each task.

Rows marked “+ ADC” in the main results table enable the ADC for the low-bit analog path; rows without “+ ADC” disable the ADC step and therefore measure ordinary digital low-bit inference quality. Accuracy is reported as a percentage (higher is better), and Avg denotes the unweighted mean across the eight downstream tasks.

5.3 Main Results

Table 3 reports the final end-to-end evaluation of Llama-3.2-1B under the Signed readout ADC simulator. The INT8 rows use W8A8 quantisation and the INT4 rows use W4A4 quantisation. All rows are produced by a single unified calibration recipe (1024 samples, Stage A + Stage B), so that the INT8 and INT4 entries are directly comparable and can

Table 2: Downstream benchmarks used for accuracy evaluation. All tasks are scored with the lm-evaluation-harness in multiple-choice log-likelihood mode. “Metric” indicates whether the reported number is length-normalised accuracy (`acc_norm`) or plain accuracy (`acc`); harness defaults are used in either case. “ n -shot” is the number of in-context demonstrations prepended to each prompt.

Task	Domain	Size	n -shot	Metric
HellaSwag	Commonsense sentence completion	10 042	0	<code>acc_norm</code>
MMLU	57 academic / professional subjects	14 042	5	<code>acc</code>
WinoGrande	Commonsense coreference	1 267	5	<code>acc</code>
ARC-Easy	Grade-school science (easy)	2 376	0	<code>acc_norm</code>
ARC-Challenge	Grade-school science (hard)	1 172	0	<code>acc_norm</code>
PIQA	Physical commonsense reasoning	1 838	0	<code>acc_norm</code>
OpenBookQA	Elementary science (open-book)	500	0	<code>acc_norm</code>
BoolQ	Yes/No reading comprehension	3 270	0	<code>acc</code>

be fed through the same lm-evaluation-harness sweep. As a consequence, the INT8 + ADC PTQ row (16.85 WikiText-2) is slightly worse than the best INT8 calibration ablation in Table 4 (14.41 at 128 samples, WikiText-only); the paragraph after Table 4 discusses the difference between the two protocols.

Rows labelled simply “INT8 PTQ” and “INT4 PTQ” are ordinary digital low-bit baselines. Weights and activations are quantised to the indicated precision and tiled integer matrix–vector multiplication is used, but the ADC step is disabled. These rows therefore represent the quality of the corresponding low-bit model in an idealised digital or infinite-resolution-readout setting.

Rows marked “+ ADC” enable the finite-resolution ADC ($b_a = 8$, fixed step δ) after each analog accumulation. These rows correspond to the analog deployment setting studied in this thesis. The table should be read as a comparison between final selected configurations for each deployment mode; matched ADC on/off ablations for individual checkpoints are reported in the following subsections.

Perplexity is reported on WikiText-2 and C4, where lower is better. Downstream task scores are accuracies, where higher is better. The detailed ablations that justify the selected configurations are reported in the following subsections.

The first observation is that ordinary digital PTQ and ADC-enabled analog PTQ are qualitatively different settings. In the digital INT8 row, quantisation alone is almost lossless in perplexity: WikiText-2 changes from 8.68 to 8.69, and C4 changes from 13.13 to 13.15. This indicates that the W8A8 digital path is not the limiting factor for this model. Once the ADC is enabled, however, the same bit-width regime becomes much worse: INT8 + ADC PTQ reaches WikiText-2 PPL 16.85 and C4 PPL 29.56, and the downstream average drops to 44.97. Thus, the ADC introduces an additional error source that is not captured by standard digital PTQ.

The same pattern is stronger at INT4. The ordinary digital INT4 PTQ baseline remains usable — WikiText-2 PPL 14.13, C4 PPL 22.83, average downstream accuracy 48.64 —

Table 3: Main results on Llama-3.2-1B.

Method	PPL ($\downarrow$)		Accuracy % ($\uparrow$)								
	Wiki	C4	HSwag	MMLU	WinGr	ARC-E	ARC-C	PIQA	OBQA	BoolQ	Avg
bfloat16	8.68	13.13	64.19	32.15	63.06	61.83	36.77	74.70	37.40	63.91	54.25
INT8 PTQ	8.69	13.15	64.17	31.89	63.38	69.82	39.16	75.79	39.20	64.22	55.95
INT8 + ADC PTQ	16.85	29.56	45.04	26.83	56.59	48.44	28.84	64.36	30.00	59.63	44.97
INT4 PTQ	14.13	22.83	51.98	24.86	55.25	60.44	33.02	69.48	32.00	62.11	48.64
INT4 + ADC PTQ	26.66	45.62	40.03	24.41	53.43	42.63	26.02	61.70	27.00	60.67	41.99
INT4 + ADC + LoRA	14.03	23.20	49.96	25.72	53.67	54.63	29.27	68.06	34.60	60.67	47.07

while the best pure INT4 + ADC PTQ configuration reaches WikiText-2 PPL 26.66, C4 PPL 45.62, and average downstream accuracy 41.99. This comparison is conservative because the ADC row is already calibrated for the ADC-enabled setting. Even after ADC-aware calibration, the finite-resolution output lattice still causes a large degradation relative to ordinary digital INT4 inference.

The second observation is that ADC-aware PTQ makes INT4 analog inference usable, but it does not close the gap to full precision. The best pure INT4 + ADC PTQ result is still about $3.1\times$ worse than the bfloat16 WikiText-2 baseline:

$$\frac{26.66}{8.68} \approx 3.1.$$

On C4, the gap is similarly large: perplexity increases from 13.13 in full precision to 45.62 under INT4 + ADC PTQ. This confirms the main limitation of transform-based PTQ in this setting. Rotations, propagated calibration, diagonal scaling, and staged training can reshape the pre-ADC signal and strongly reduce the catastrophic collapse observed in the naive baseline, but they still operate before the fixed ADC lattice. Once the analog output has been discretised, the remaining error cannot be fully removed by further pre-ADC reshaping alone.

The third observation is that a small post-ADC low-rank correction recovers a large fraction of the remaining perplexity gap. Adding the post-ADC LoRA branch reduces WikiText-2 perplexity from 26.66 to 14.03, a relative reduction of 47.4%. Measured as gap closure with respect to the bfloat16 baseline, this recovers approximately 70% of the WikiText-2 degradation introduced by INT4 + ADC PTQ:

$$\frac{(26.66 - 8.68) - (14.03 - 8.68)}{26.66 - 8.68} \approx 0.70.$$

The same pattern holds on C4. Perplexity decreases from 45.62 to 23.20, which corresponds to approximately 69% closure of the gap to the bfloat16 C4 baseline:

$$\frac{(45.62 - 13.13) - (23.20 - 13.13)}{45.62 - 13.13} \approx 0.69.$$

This result is particularly important because the LoRA correction is trained on WikiText-2, while C4 is held out. The C4 improvement therefore suggests that the correction is not merely memorising the calibration distribution, but learns a residual compensation that transfers to a different text corpus.

The downstream results show a more nuanced but consistent picture. Post-ADC LoRA increases the average downstream accuracy from 41.99 to 47.07, recovering 5.08 points over the INT4 + ADC PTQ baseline. Relative to the ordinary digital INT4 PTQ reference, this recovers most of the ADC-induced downstream drop:

$$\frac{47.07 - 41.99}{48.64 - 41.99} \approx 0.76.$$

The largest gains appear on HellaSwag (40.03 $\rightarrow$ 49.96), ARC-Easy (42.63 $\rightarrow$ 54.63), PIQA (61.70 $\rightarrow$ 68.06), and OpenBookQA (27.00 $\rightarrow$ 34.60). MMLU and WinoGrande improve only slightly, and BoolQ is unchanged. Thus, the post-ADC correction gives its strongest and most consistent recovery in perplexity, while downstream multiple-choice accuracy is recovered only partially and in a task-dependent way. This is expected because the LoRA branch is trained with next-token cross-entropy and teacher distillation, not directly on the downstream tasks.

Finally, Table 3 gives an important hardware-oriented comparison across bit-widths. INT4 + ADC + LoRA does not outperform the ordinary digital INT8 PTQ baseline; the latter is essentially a no-ADC, near-full-precision reference. The relevant analog comparison is instead INT8 + ADC PTQ. Against that baseline, INT4 + ADC + LoRA achieves lower perplexity on both WikiText-2 (14.03 vs. 16.85) and C4 (23.20 vs. 29.56), and higher average downstream accuracy (47.07 vs. 44.97), despite using 4-bit weights and activations in the analog path. It also improves over INT8 + ADC PTQ on six of the eight downstream tasks, while remaining worse on MMLU and WinoGrande. The conclusion is therefore not that INT4 analog inference beats ideal digital INT8 inference, but that post-ADC correction can make a 4-bit ADC-enabled analog path competitive with, and in this table stronger than, an 8-bit ADC-enabled PTQ baseline.

5.4 Signed Readout: Ablation Results

All results below use the *signed readout*: the analog array is modelled as producing a signed integer accumulation $y_{\text{int}} \in [-Mq_xq_w, +Mq_xq_w]$, and the ADC uses a symmetric signed output range with the hardware constants from Table 1. Throughout the remainder of the thesis we use **Signed** as a shorthand for this signed-accumulation readout and **Unsigned** for the single-read unsigned shift-subtract readout introduced in Sec. 5.5. All tables in this subsection report WikiText-2 perplexity unless stated otherwise; the bfloat16 full-precision baseline is 8.68 (see Sec. 5.2).

5.4.1 INT8: Effect of Calibration Objective

Table 4 compares four INT8 configurations at 128 calibration samples, isolating the contribution of block-wise propagation and the bin-center auxiliary loss.

The bfloat16 baseline reports the full-precision reference for comparison; no quantisation is applied.

The Baseline configuration ($\alpha = 0$) uses the standard FlatQuant teacher–student calibration with clean FP teacher inputs and no ADC-aware terms in the loss; the ADC is enabled only at evaluation. The objective reduces to

$$\mathcal{L}_{\text{FP}} = \|\hat{B}(x^*) - B_{\text{FP}}(x^*)\|_2^2, \quad (46)$$

where $\hat{B}$ is the quantised student block and x^* is the clean teacher activation from the FP model.

The Bin-center variant ($\lambda = 0.1$) adds an auxiliary loss to $\mathcal{L}_{\text{FP}}$ that penalises pre-ADC accumulated values lying close to ADC bin boundaries:

$$\mathcal{L}_{\text{bin}} = \mathbb{E} \left[\cos^2 \left(\pi \frac{y_{\text{int}}}{\delta} \right) \right], \quad \mathcal{L} = \mathcal{L}_{\text{FP}} + \lambda \mathcal{L}_{\text{bin}}. \quad (47)$$

The cosine term is one at integer values of y_{int}/δ (bin boundaries) and zero at half-integer values (bin centres), so minimising it pushes pre-ADC values toward bin centres, where small calibration perturbations are less likely to flip the post-ADC code.

Figure 6 illustrates the shape of $\mathcal{L}_{\text{bin}}$. The penalty is maximal at integer values of $z = y_{\text{int}}/\delta$, which are exactly the bin-boundary positions where $\lfloor z \rfloor$ flips and any small calibration perturbation can change the post-ADC code. It is zero at half-integer values — the bin-centre positions — which are the most robust quantisation states under perturbation. Minimising $\mathcal{L}_{\text{bin}}$ therefore pushes pre-ADC accumulators toward bin centres, where the ADC output is locally stable.

The Propagated configuration ($\alpha = 1$) replaces the clean teacher input at each block with the ADC-quantised output $\hat{x}$ of the preceding block, matching the calibration-time input distribution to the deployment-time distribution. The objective becomes

$$\mathcal{L}_{\text{ADC}} = \|\hat{B}(\hat{x}) - B_{\text{FP}}(x^*)\|_2^2. \quad (48)$$

The Prop + bin-center row combines both modifications: the propagated objective $\mathcal{L}_{\text{ADC}}$ replaces $\mathcal{L}_{\text{FP}}$ as the reconstruction term, and the bin-centre regulariser is added on top, giving

$$\mathcal{L} = \mathcal{L}_{\text{ADC}} + \lambda \mathcal{L}_{\text{bin}}. \quad (49)$$

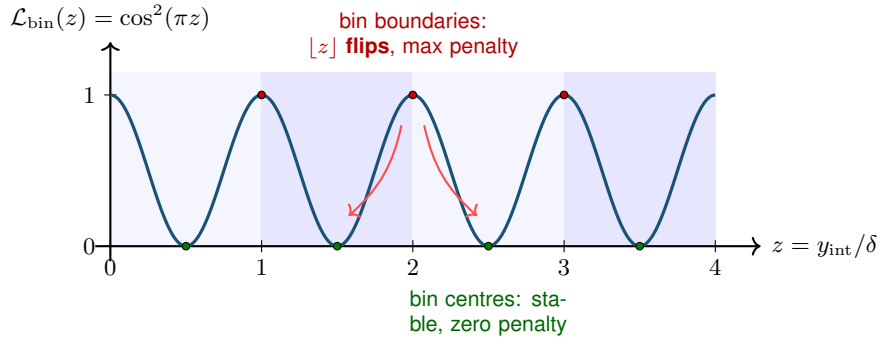


Figure 6: The bin-centre auxiliary loss

Propagation alone reduces ADC PPL by 47% (28.86 to 14.55) with no change to dead-rate, confirming that the remaining gap is ADC resolution rather than the dead zone. The bin-center loss has no effect on its own but gives a marginal improvement (+0.14 PPL) on top of propagation.

Relation to Table 3. Table 4 is a WikiText-only calibration-objective ablation at 128 samples with a single calibration stage; its purpose is to isolate the effect of propagation and the bin-center auxiliary loss. The 14.41 reported here is therefore the best WikiText-2

Table 4: INT8 calibration study on Llama-3.2-1B (128 samples, $\delta \approx 2016$).

Configuration	no-ADC PPL	ADC PPL
bfloat16 baseline	—	8.68
Baseline ($\alpha = 0$)	10.01	28.86
Bin-center ($\lambda = 0.1$)	10.01	28.86
Propagated ($\alpha = 1$)	11.01	14.55
Prop + bin-center	11.00	14.41

PPL achievable for this restricted setup, not a deployable checkpoint. The main results in Table 3 use a different protocol: 1024 calibration samples and the full Stage A + Stage B pipeline shared across all INT8/INT4 configurations so that the comparison across bit-widths is on a unified recipe. Under that recipe the INT8 + ADC PTQ checkpoint reaches WikiText-2 PPL 16.85. The two numbers are not contradictory: the staged recipe trades a small amount of INT8 WikiText-2 quality for the cross-configuration consistency required to compare bit-widths and to run the lm-evaluation-harness sweep, while the calibration-objective conclusions of Table 4 (propagation matters, the bin-center loss is a marginal addition) carry over to the deployed pipeline.

5.4.2 INT4: Sample Count and Propagation

At INT4 ($b_x = b_w = 4$) the ADC step size drops from $\delta \approx 2016$ at INT8 to $\delta \approx 6.12$ — two orders of magnitude finer. The no-ADC baseline reaches WikiText-2 perplexity 15.41, but enabling the ADC step blows it up to 2354.86: a $\times 153$ gap, against $\times 2.9$ at INT8. The experiments below sweep the dominant levers — sample count, propagation, transform initialisation, and decoupled training — on Llama-3.2-1B; Table 5 summarises the runs.

Propagation was the dominant lever at INT8, so it is the natural first try. At 128 samples it drops ADC perplexity to 203.89 but pushes no-ADC perplexity from 15.41 to 40.28: the transforms optimise for ADC-distorted inputs at the cost of clean ones. The gap narrows to $\times 5.1$ but the absolute ADC perplexity remains unusable, which already suggests 128 samples are too few for propagated INT4 calibration to generalise.

Two orthogonal modifications reported to help in similar pipelines were tested next. Replacing the random orthogonal Kronecker init with a Hadamard matrix [2], both alone and with propagation, produces numbers within noise of the random-init counterparts at the same budget (Table 5); the bottleneck is not where the optimiser starts. A decoupled scheme — training transforms at INT8 weights but evaluating under the INT4 ADC — gives the best no-ADC perplexity of the sweep (11.64, close to INT8) but an ADC perplexity of 276.25 ($\times 24$ gap), because transforms tuned for the INT8 loss landscape do not match the much finer INT4 δ .

The dominant lever is in fact the sample count. Raising the calibration set to 512 samples and rerunning propagated calibration yields no-ADC perplexity 34.25 and ADC perplexity **40.63** ($\times 1.2$ gap) — a five-fold drop in ADC perplexity over the 128-sample propagated run, with the gap now within a few percent of the no-ADC measurement. As a sanity check, the non-propagated baseline rerun at 1024 samples gives no-ADC 19.80 and ADC 56.83, a $\times 42$ ADC improvement over the 128-sample baseline driven purely by data. Propagation still helps on top, but the experiment confirms that low-bit calibration is funda-

mentally sample-hungry: at $b_w = b_x = 4$ the calibration objective sees far fewer effective gradient signals per token, and more data is needed to span the activation distribution well enough for the transforms to converge.

Table 5: INT4 sample count and propagation sweep on Llama-3.2-1B ($\delta \approx 6.12$). Gap is the ratio ADC PPL / no-ADC PPL on WikiText-2.

Config	Samples	no-ADC PPL	ADC PPL	Gap
Baseline	128	15.41	2354.86	$\times 153$
Propagated ($\alpha = 1$)	128	40.28	203.89	$\times 5.1$
Hadamard init	128	15.08	2080.51	$\times 138$
Hadamard init + prop	128	34.73	178.86	$\times 5.1$
Decoupled (train w8, eval w4)	128	11.64	276.25	$\times 24$
Propagated ($\alpha = 1$)	512	34.25	40.63	$\times 1.2$
Baseline	1024	19.80	56.83	$\times 2.9$

5.4.3 INT4: α -Mixing Sweep (1024 Samples)

Section 5.4.2 established that 1024 calibration samples and propagation are necessary for INT4 to generalise. The natural follow-up question is what mixing weight α in the dual-forward objective $\mathcal{L}(\alpha) = (1-\alpha) \mathcal{L}_{\text{FP}} + \alpha \mathcal{L}_{\text{ADC}}$ best balances clean and ADC-propagated calibration. Table 6 sweeps α from 0 (clean teacher only) to 1 (fully propagated) at 1024 samples, with three random calibration seeds per cell.

Table 6: INT4 α -mixing sweep. 1024 calibration samples. Values are mean $\pm 1\sigma$ over 3 random calibration seeds.

Config	α	no-ADC PPL	ADC PPL	Gap
Baseline	0.00	19.80 ± 0.5	56.83 ± 2.5	$\times 2.87$
Prop $\alpha = 0.25$	0.25	21.65 ± 0.5	32.35 ± 1.0	$\times 1.49$
Prop $\alpha = 0.5$	0.50	24.24 ± 0.6	31.22 ± 0.9	$\times 1.29$
Prop $\alpha = 0.75$	0.75	27.46 ± 0.8	35.92 ± 1.2	$\times 1.31$
Prop $\alpha = 1.0$ (full)	1.00	30.32 ± 1.0	40.20 ± 1.5	$\times 1.32$

Figure 7 visualises the sweep. The minimum ADC perplexity is attained at $\alpha = 0.5$ (31.22, gap $\times 1.29$), which is also the best gap ratio in the sweep. Smaller values under-weight the ADC regime — the transforms shape pre-ADC distributions for an objective that does not see the $\lfloor \cdot \rfloor$ step — while larger values amplify gradient noise from the floor operator and steadily degrade no-ADC quality without further ADC gains.

5.4.4 INT4: Diagonal Placement, MLP Decomposition, and Staged Training

Section 5.4.2 converged to a recipe (1024 samples, propagation with $\alpha = 0.5$) that closes the gap to $\times 1.29$ but is still bounded at ADC PPL ≈ 31 . The bounded per-channel scaling component, inherited from FlatQuant-style equivalent transformations [23] and used here in an ADC-aware setting, is the next degree of freedom on top of the Kronecker

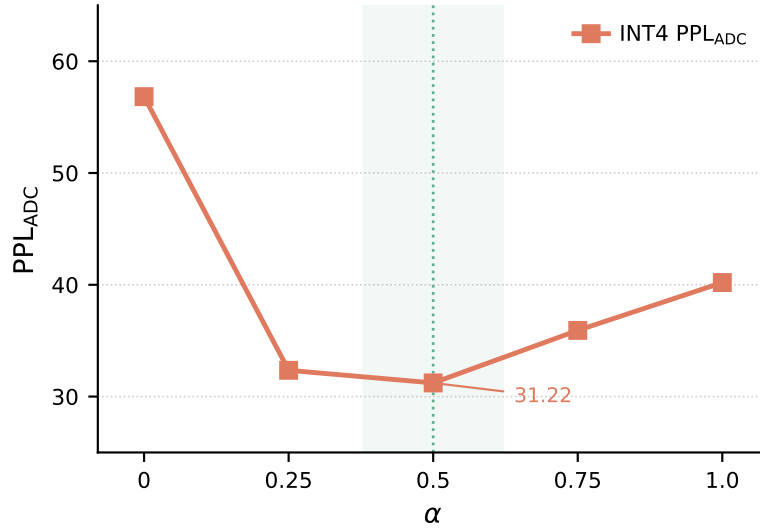


Figure 7: α -mixing sweep at 1024 calibration samples. ADC PPL is minimized at $\alpha = 0.5$; smaller values under-weight the ADC regime, larger values amplify gradient noise from the floor operator and degrade no-ADC PPL.

transforms: a per-channel scale $D = \text{diag}(d_1, \dots, d_c)$ with $d_i \in [10^{-4}, 10]$ that absorbs into the surrounding weights at deployment. The model has *two* natural positions for such a diagonal — one before the attention sub-layer (acting on $q/k/v$ inputs, plus a second instance before o) and one before the MLP sub-layer (acting on gate/up inputs, plus a second before down). Table 7 ablates these two placements and decomposes them further.

Reading the first block of Table 7 reveals a consistent pattern that is also summarised in Figure 8. The MLP diagonal acts primarily on the transform-quality axis: enabling it alone drops no-ADC PPL to 19.37 (close to its best achievable value), but leaves the ADC gap wide (31.65 ADC PPL).

The attention diagonal behaves dually: enabling it alone barely moves no-ADC PPL (24.98, worse than no diagonal) but produces a comparatively small ADC gap (29.41). Intuitively, the MLP diagonal reshapes the per-channel variance of the activations that feed the feed-forward weight matrices — this is where INT4 rounding errors are largest, so any reduction directly improves digital inference quality. The attention diagonal, by contrast, mostly affects the magnitude of pre-ADC accumulators inside the GQA block, which controls how often a tile’s y_{int}/δ falls into the dead-zone or saturates — it has little leverage on digital quality but is essential for ADC robustness. Configurations that activate only one of the two leave significant perplexity on the table; the two diagonals are complementary, not redundant.

Inside the MLP diagonal: down_proj dominates. The MLP diagonal can be further decomposed into the diagonal that precedes the gate/up projections and the one that precedes down . The second block of Table 7, visualised in Figure 9, shows that the two are far from equal: the up_gate diagonal alone yields no-ADC PPL 23.22 with a wide gap (39.59 ADC), while the down diagonal alone is already close to the full-MLP setting (20.42/31.26). Mechanistically this is exactly what one expects: down_proj is the last linear layer before the residual addition in each block, and its output is what the next block’s

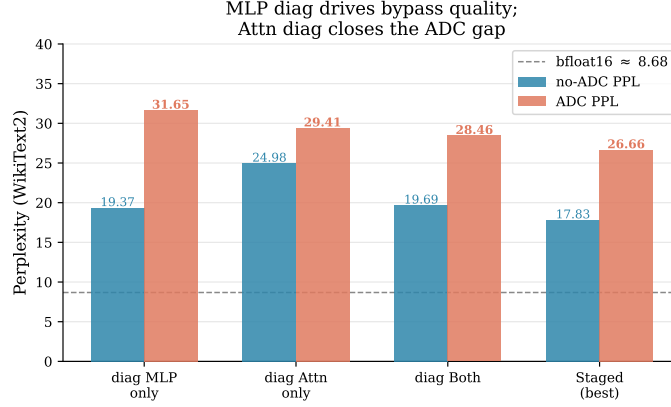


Figure 8: Roles of the MLP and attention diagonals. The MLP diagonal drives no-ADC PPL improvements (transform quality); the attention diagonal closes the no-ADC-to-ADC gap (robustness to ADC noise). Either alone leaves significant perplexity on the table; staged training combines both.

Table 7: INT4 diagonal placement and staged training. 1024 samples, $\alpha = 0.5$. Values are mean $\pm 1\sigma$ over 3 random calibration seeds, except the final staged row, which reports the selected checkpoint used in Table 3.

Config	no-ADC PPL	ADC PPL	Note
Both diag (control)	19.69 ± 0.5	28.46 ± 0.8	joint diag baseline
Attn diag only	24.98 ± 0.6	29.41 ± 0.8	worse no-ADC PPL
MLP diag only	19.37 ± 0.4	31.65 ± 1.0	best no-ADC PPL, wide gap
MLP diag + layer-wise α	19.00 ± 0.4	28.66 ± 0.7	best no-ADC PPL overall
Both diag + layer-wise α	19.39 ± 0.4	28.55 ± 0.7	no gain over flat α
up_gate diag only	23.22 ± 0.7	39.59 ± 1.4	weak
down diag only	20.42 ± 0.5	31.26 ± 0.9	main contributor
Staged: MLP diag $\rightarrow$ attn diag	17.83	26.66	best PTQ (Table 3)

tiles will read through the ADC, so a per-channel scaling here directly controls the conditioning of the most consequential pre-ADC signal in the network. The *up/gate* diagonal acts on activations that have already passed through the FlatQuant rotations and the SwiGLU non-linearity, where the rotation has already absorbed most of the per-channel variance — adding another diagonal here is largely redundant.

Because the MLP and attention diagonals optimise different things, training them jointly with a single α creates gradient interference: gradients that would push the MLP diagonal toward a clean-input optimum compete with gradients pushing the attention diagonal toward ADC-robust scales. The control row in Table 7 (*Both diag*, ADC 28.46) sits between the two single-diagonal extremes, confirming that the joint optimum is a compromise rather than a synthesis.

Staged training resolves this conflict by fitting the two diagonals in separate phases, while keeping the Kronecker basis trainable throughout (Listing 5).

Stage A (30 epochs at the base learning rate η) enables the MLP diagonal $D^{(m)}$ and freezes the attention diagonal $D^{(a)}$. The Kronecker factors of all seven projections are optimised under the pure FP reconstruction loss ($\alpha = 0$), so the MLP scaling and the underlying rotations converge on a clean, low-variance objective without seeing any ADC noise; this drives the no-ADC PPL down to its best achievable value.

Stage B (10 epochs at 0.1η) freezes $D^{(m)}$ at its Stage-A value and enables $D^{(a)}$, switching to the mixed objective with $\alpha = 0.5$ so that gradients flow through the floor operator. The Kronecker factors remain trainable, but the $10\times$ smaller learning rate means the MLP basis is only fine-tuned around its Stage-A optimum. Effectively the MLP basis is held in place while the attention diagonal is fitted under ADC-propagated inputs — a small, well-conditioned fine-tune of the attention block under propagated inputs — the very signal the attention diagonal is supposed to reshape.

This schedule achieves the best pure-PTQ result in our sweep: no-ADC PPL 17.83 and ADC PPL 26.66, lower than the single-stage joint baseline on both axes and with a tighter spread across calibration seeds. The improvement is not an additional capacity gain over single-stage training — the parameter set is identical — but rather a direct consequence of separating the two sub-systems’ optimisation trajectories so that neither pulls the other away from its natural optimum.

5.4.5 INT4: Stochastic Propagation

The deterministic $\alpha = 0.5$ objective optimises an expectation over a fixed, equal mixture of clean and ADC-propagated forwards. Section 5.4.4 showed that this point is the best gap-ratio operating point of the deterministic family; the natural follow-up is whether replacing the constant α by a random variable α_t sampled per batch can improve any of the three measurements: no-ADC PPL, ADC PPL, or the gap. Two stochastic schedules were compared with the deterministic baseline.

The first is a deterministic mixture at $\alpha = 0.5$: a single dual-forward objective at fixed weight, evaluated every batch,

$$\mathcal{L}_{\text{det}} = \frac{1}{2} \mathcal{L}_{\text{FP}}(B) + \frac{1}{2} \mathcal{L}_{\text{ADC}}(B). \quad (50)$$

Every gradient step sees both losses simultaneously. This is the reference configuration from Section 5.4.3.

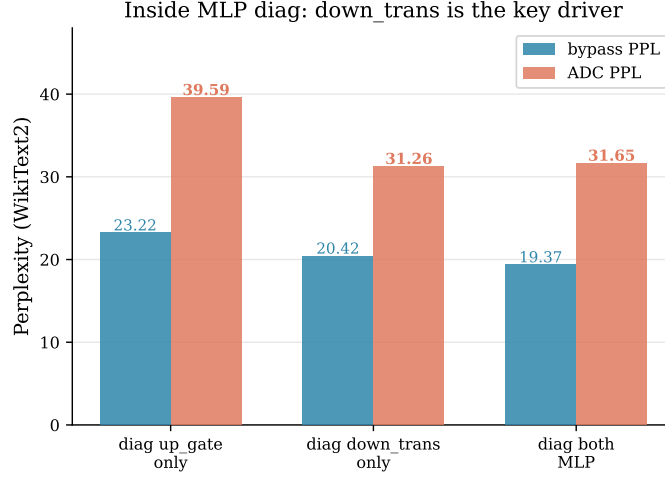


Figure 9: MLP-diagonal decomposition. `down_proj` contributes nearly all of the no-ADC PPL improvement; the `up/gate` diagonal contributes little, since `down_proj` sits immediately before the next ADC accumulation and shapes the signal that is read.

The second replaces this constant weight with Bernoulli switching at $p = 0.5$: at each batch t , a single Bernoulli draw selects which forward pass is executed,

$$\alpha_t \sim \text{Bernoulli}(0.5), \quad \mathcal{L}_{\text{Bern}}(t) = \begin{cases} \mathcal{L}_{\text{FP}}(B), & \alpha_t = 0, \\ \mathcal{L}_{\text{ADC}}(B), & \alpha_t = 1. \end{cases} \quad (51)$$

Only one forward through the block is performed per batch (half the compute of the deterministic variant), and in expectation $\mathbb{E}_t[\mathcal{L}_{\text{Bern}}] = \mathcal{L}_{\text{det}}$. This is the QDrop-style schedule [36]: clean and ADC-propagated losses appear in alternation rather than averaged per batch.

The third uses continuous $\text{Beta}(2, 2)$ sampling: at each batch a mixing weight is drawn from a symmetric Beta distribution concentrated around 0.5, and the dual-forward objective is evaluated at that weight,

$$\alpha_t \sim \text{Beta}(\beta, \beta) \ (\beta = 2), \quad \mathcal{L}_{\text{Beta}}(t) = (1 - \alpha_t) \mathcal{L}_{\text{FP}}(B) + \alpha_t \mathcal{L}_{\text{ADC}}(B). \quad (52)$$

With $\beta = 2$ the density is peaked at 0.5 with variance $1/20$, so most batches see a near-equal mixture and occasional batches lean slightly toward one extreme. This is a smoother stochastic variant of the deterministic schedule: it preserves the dual-forward structure but adds light noise to the mixing weight.

Each schedule was evaluated in both the flat setting (single-stage training, both diagonals jointly) and the staged setting from Section 5.4.4 (Stage A: MLP only, $\alpha = 0$; Stage B: attention added, with the stochastic objective applied in Stage B). Table 8 reports both configurations.

Two patterns emerge consistently across both modes. First, Bernoulli switching gives the best *no-ADC* perplexity in each mode (19.25 flat, 16.94 staged) but the worst ADC gap ($\times 1.84$ and $\times 1.92$). This follows directly from the schedule’s structure: in batches with $\alpha_t = 0$ the attention transforms see no ADC signal and drift toward configurations that

Table 8: INT4 stochastic propagation. 1024 samples. “Mode” flat = single-stage joint training; staged = Stage A/B schedule from Table 7 with the stochastic objective used in Stage B.

Schedule	Mode	no-ADC PPL	ADC PPL	Gap
Deterministic $\alpha = 0.5$	flat	24.24	31.22	$\times 1.29$
Bernoulli ($p = 0.5$)	flat	19.25	35.38	$\times 1.84$
Beta(2, 2)	flat	22.90	31.40	$\times 1.37$
Deterministic $\alpha = 0.5$	staged	17.83	26.66	$\times 1.50$
Bernoulli ($p = 0.5$)	staged	16.94	32.44	$\times 1.92$
Beta(2, 2)	staged	17.16	27.82	$\times 1.62$

minimise the clean loss without any pressure to be ADC-robust, while in batches with $\alpha_t = 1$ the optimiser has to compensate for that drift. The two regimes partially cancel, but the residual drift toward clean-loss optima improves no-ADC quality at the cost of ADC robustness.

Second, Beta(2, 2) behaves almost identically to the deterministic schedule. Because the Beta(2,2) density is peaked near 0.5 with low variance, the per-batch mixing weights cluster tightly around the deterministic value, and the resulting trained transforms differ from the deterministic baseline only by an amount commensurate with run-to-run noise. The marginal ADC difference in the staged setting (27.82 vs 26.66) is within one standard deviation across calibration seeds and does not justify the additional per-batch sampling.

Taken together, the stochastic schedules do not displace the deterministic $\alpha = 0.5$ recipe. The deterministic staged configuration remains the best PTQ result on the no-ADC-vs-ADC Pareto front: it matches the staged-Beta ADC PPL without the noisier gradients, and it dominates the staged-Bernoulli configuration on the ADC axis even though Bernoulli is better on no-ADC. We therefore retain deterministic $\alpha = 0.5$ as the default mixing schedule for all downstream experiments.

5.4.6 Post-ADC Residual LoRA: Initial Sweep

Table 10 evaluates LoRA adapters on top of the best PTQ checkpoint (the staged configuration, ADC PPL 26.66), varying rank and target projection sets. The corresponding parameter overhead for each configuration is reported in Table 9: rank-4 adapters on all seven projections add 2.82M parameters (0.228% of the base model), doubling the rank doubles this count, and restricting the targets to `down` and `o_proj` reduces the overhead to 0.92M parameters (0.074%).

Table 9: Post-ADC LoRA overhead for Llama-3.2-1B ($d_{\text{model}} = 2048$, $d_{\text{ffn}} = 8192$, $d_{\text{kv}} = 512$, 16 layers).

Target	rank	added params	% of model	extra MACs/token
all 7 projections	4	2.82M	0.228%	2.82M
all 7 projections	8	5.64M	0.456%	5.64M
down + o_proj	4	0.92M	0.074%	0.92M

Table 10: LoRA initial sweep. Base PTQ: no-ADC 17.83, ADC 26.66.

Config	Targets	no-ADC PPL	ADC PPL	Gap
No LoRA (base PTQ)	—	17.83	26.66	$\times 1.50$
LoRA $r = 4$, down	1 proj	19.33	16.32	$\times 0.84$
LoRA $r = 8$, down	1 proj	19.17	15.79	$\times 0.82$
LoRA $r = 4$, down+o	2 proj	18.68	15.63	$\times 0.84$
LoRA $r = 4$, all 7 proj	7 proj	17.73	16.53	$\times 0.93$

Across all LoRA configurations ADC PPL falls below no-ADC PPL ($\text{gap} < 1$), confirming that the adapter learns to specifically correct ADC noise. Adding o_proj to down_proj gives the best ADC PPL (15.63); covering all 7 projections gives the best no-ADC PPL (17.73) at the cost of a slightly wider gap.

5.4.7 Post-ADC Residual LoRA: Ablation Study

Table 11 reports the full ablation across rank, loss, target projections, layer coverage, and adapter placement (all runs use $r = 4$ unless stated; the **Targets** column makes the set of projections explicit for each row).

Table 11: LoRA ablation study. Llama-3.2-1B, INT4. Targets: down+o = down-projection of MLP plus output projection of attention; all 7 = all seven projections per block.

Axis	Setting	Targets	ADC PPL
Seed (CE)	seed 1	down+o	15.79
	seed 2	down+o	15.88
	seed 3	down+o	15.93
Rank (CE)	$r = 1$	down+o	15.90
	$r = 2$	down+o	15.60
	$r = 4$	down+o	15.57
	$r = 8$	down+o	15.24
Loss $\times$ Targets	CE only	down+o	15.63
	CE + KL	down+o	14.33
	CE only	all 7	16.68
	CE + KL	all 7	14.03
Layer coverage (CE)	first 8 layers	down+o	16.64
	all 16 layers	down+o	15.63
	last 8 layers	down+o	20.08
Placement (CE)	post-ADC (proposed)	down+o	15.63
	pre-ADC	down+o	105.63 (<i>diverged</i>)

Rank saturates at $r = 4$ (only 0.33 PPL gap to $r = 8$). CE+KL outperforms CE alone by

1.30 PPL with down+o targets (15.63 $\rightarrow$ 14.33) and by 2.65 PPL with all-7 targets (16.68 $\rightarrow$ 14.03); the FP teacher signal is most useful when the adapter covers more projections. Earlier layers carry more accumulated ADC error: first-8 (16.64) beats last-8 (20.08). Pre-ADC placement diverges (ADC PPL 105.6) because the correction is itself quantized by the fixed δ . The best single configuration is all 7 targets, CE+KL, $r = 4$: ADC PPL 14.03, beating the deployed INT8 + ADC PTQ checkpoint (16.85, Table 3).

5.4.8 Post-ADC Residual LoRA: Generalization to C4

Table 12 evaluates the top-3 configurations on both WikiText-2 and C4 to test out-of-domain transfer.

Table 12: LoRA generalization. WikiText-2 and C4 perplexity.

Config	Wiki no-ADC	Wiki ADC	C4 no-ADC	C4 ADC
PTQ (no LoRA)	17.83	26.66	28.58	45.62
$r = 8$, down+o, CE	18.51	15.47	30.48	29.39
$r = 4$, down+o, CE+KL	19.05	14.27	29.37	23.88
$r = 4$, all 7, CE+KL	18.61	14.03	29.14	23.20

CE+KL configurations reduce C4 ADC PPL by roughly half relative to the PTQ baseline (45.62 $\rightarrow$ 23.20) despite being calibrated solely on WikiText-2. CE-only ($r = 8$, down+o) shows much weaker C4 generalization (29.39 vs. 23.20). The KL teacher signal is therefore the dominant factor for cross-domain transfer. All-7 CE+KL is the best overall checkpoint on both datasets.

5.5 Unsigned Shift-Subtract Readout: A Second Hardware Setting

All experiments described so far use the Signed readout: the analog crossbar accumulates a signed integer dot product $y_{\text{int}} \in [-Mq_xq_w, +Mq_xq_w]$ and the ADC reads both positive and negative values. A physically important alternative is an *unsigned shift-subtract readout*. Here the signed INT4 activation and weight codes are first shifted to unsigned codes in $[0, 15]$, the array performs a single non-negative MVM, the ADC quantises the unsigned accumulated value, and the deterministic zero-point terms are subtracted digitally after readout. We refer to this setting as **Unsigned**. Real crossbar hardware is often constrained to non-negative currents, making the Unsigned setting the more realistic operating regime. The two readouts share the same bit-width $b_a = 8$ and tile size $M = 256$, but they differ in the ADC input range and in how the signed dot product is recovered, which changes δ by more than a factor of two and requires a different calibration objective. The Unsigned setting is therefore treated as a self-contained experiment series with its own baselines, calibration, and ablation structure.

5.5.1 Unsigned Shift-Subtract ADC Algorithm

Because the crossbar accepts only $[0, 2^b - 1]$ codes, the Unsigned readout uses an offset-binary 4-bit codebook $\hat{x}^u, \hat{w}^u \in [0, 15]$ with zero point $z = 2^{b_x-1} = 8$. The corresponding centered integer codes $\hat{x} = \hat{x}^u - z$ and $\hat{w} = \hat{w}^u - z$ therefore lie in $[-8, 7]$ — a 16-level range used only for the Unsigned hardware path, in place of the symmetric $q_{\text{max}} = 7$

convention of Table 1 used by the Signed reference. The deterministic zero-point terms are subtracted digitally after the ADC; as Step 4 shows, this correction is algebraically exact for the shift terms, while ADC rounding and clipping error remains. The algorithm proceeds in five steps.

Step 1 — Shift to unsigned. Each signed code is offset by the zero-point $z = 2^{b_x-1} = 8$:

$$\hat{x}^u = \hat{x} + 8 \in [0, 15], \quad \hat{w}^u = \hat{w} + 8 \in [0, 15].$$

Step 2 — Hardware MVM. The crossbar computes the non-negative dot product over one tile of width $M = 256$:

$$y^u = \langle \hat{x}^u, \hat{w}^u \rangle \in [0, M(2^{b_x} - 1)(2^{b_w} - 1)] = [0, 57600].$$

Because $y^u \geq 0$ always, a single unsigned MVM suffices with no four-quadrant splitting.

Step 3 — ADC floor. Because the unsigned codes span the range $[0, 2^{b_a} - 1]$ rather than the symmetric $[-2^{b_a-1}, 2^{b_a-1} - 1]$ of the Signed readout, the unsigned step size is:

$$\delta_u = \frac{M(2^{b_x} - 1)(2^{b_w} - 1)}{(2^{b_a} - 1)k} = \frac{256 \cdot 15 \cdot 15}{255 \cdot 16} \approx 14.1 \quad (k = 16),$$

and the quantized output is $\hat{y}^u = \delta_u \cdot \text{clip}(\lfloor y^u / \delta_u \rfloor, 0, 2^{b_a} - 1)$. Note that $\delta_u \approx 14.1$ is more than twice the Signed $\delta \approx 6.12$ at the same k , reflecting the fact that the same number of ADC bins now covers a range roughly $2\times$ wider in absolute terms.

Step 4 — Digital zero-point correction. Expanding the unsigned product gives $\hat{x}^u \hat{w}^u = \hat{x} \hat{w} + 8\hat{w} + 8\hat{x} + 64$, so in the absence of any ADC quantisation the unsigned shift can be undone algebraically:

$$y_{\text{int}} = y^u - 8 \sum_j \hat{w}_j^u - 8 \sum_i \hat{x}_i^u + M \cdot 64.$$

In the actual pipeline, however, the correction is applied to the ADC-quantised readout $\hat{y}^u$ rather than to y^u itself. Substituting $\hat{y}^u$ into the same expression yields

$$\tilde{y}_{\text{int}} = \hat{y}^u - 8 \sum_j \hat{w}_j^u - 8 \sum_i \hat{x}_i^u + M \cdot 64 = y_{\text{int}} + (\hat{y}^u - y^u),$$

where the residual $(\hat{y}^u - y^u)$ is the ADC rounding / clipping error. The zero-point shift is therefore removed *exactly*, but the ADC quantisation error remains uncorrected: “exact” here refers only to the algebraic shift, not to the full post-ADC accumulator. This is precisely why the Unsigned setting is harder than the Signed one — the coarser δ_u enlarges the residual $(\hat{y}^u - y^u)$, motivating the per-layer k search of Section 5.5 and the post-ADC LoRA correction of Section 4. Both summands $\sum_j \hat{w}_j^u$ and $\sum_i \hat{x}_i^u$ are cheap: the weight sum is precomputed once at load time, and the token sum is a single reduction per token.

Step 5 — Dequantisation. $y_{\text{real}} = \tilde{y}_{\text{int}} \cdot s_x \cdot s_w$, where $\tilde{y}_{\text{int}}$ is the ADC-corrupted recovered accumulator from Step 4, s_x is the per-token activation scale, and s_w is the per-channel weight scale, exactly as in the Signed setting.

5.5.2 Per-Layer k Search

Because the coarser δ_u (at global $k = 16$) imposes a heavier resolution penalty than the Signed setting, a per-layer k search is introduced that selects the ADC scaling factor independently for each projection layer.

Per-layer reconfiguration of the analog read-out is not assumed to be a universal hardware property, but such configurable setups do exist — hybrid optical-electronic architectures, for instance, let the digital controller program per-layer parameters before each layer is launched [33]. Treating k as a per-projection knob is consistent with the design space exposed by such setups; no specific accelerator is claimed to implement a per-layer k in exactly the form used here, and the parameter is treated only as a deployment-time configuration vector.

Operationally, our per-layer k search is therefore a calibration-time procedure: weights and FlatQuant transforms are kept frozen, and only the per-projection k is chosen from a fixed candidate set by minimising ADC reconstruction error against the FP teacher. The selected k values become a deployment-time configuration vector that costs nothing at inference — k is a register, not a trained parameter — but recovers a meaningful fraction of the perplexity that a single global k leaves on the table for Unsigned deployment.

After FlatQuant calibration with a global k , the pre-ADC activations y_ℓ^u are captured on the calibration set for each projection layer ℓ . For each candidate $k \in \mathcal{K} = \{4, 8, 16, 32, 64\}$ the ADC reconstruction MSE is evaluated

$$\text{MSE}_\ell(k) = \mathbb{E} \left[(Q_{\text{ADC}}^{(k)}(y_\ell^u) - y_\ell^u)^2 \right],$$

where $Q_{\text{ADC}}^{(k)}$ uses $\delta_u(k)$ from Sec. 5.5, and select

$$k_\ell^* = \arg \min_{k \in \mathcal{K}} \text{MSE}_\ell(k).$$

Because δ_u is inversely proportional to k , this procedure selects the largest k whose representable range still covers the typical magnitude of y_ℓ^u without saturating. Layers with narrow output distributions therefore receive a large k (finer δ_u , range is not needed); layers with wide distributions receive a small k (coarser δ_u but wider range, avoiding saturation). The search is run in place on the fixed PTQ checkpoint without retraining FlatQuant transforms.

5.5.3 Calibration Objective for the Unsigned Readout

FlatQuant transforms calibrated for the Signed $\delta \approx 6.12$ are mismatched to the Unsigned $\delta_u \approx 14.1$: the transforms shape the pre-ADC signal for a finer resolution than the hardware does not provide. In the Unsigned series, FlatQuant is therefore retrained with the unsigned δ_u formula as the active ADC step during calibration. All other settings (staged training, $\alpha = 0.5$, diagonal scaling) are carried over from the best Signed configuration.

5.5.4 Experiment Configurations

The following seven configurations are evaluated for Llama-3.2-1B on WikiText-2 and C4. We give each a human-readable name used in the main text and keep the original short identifier (in `typewriter`) in parentheses, since it also labels the corresponding row in Table 14.

- **bfloat16 baseline** (fp) — full-precision reference, no quantisation.
- **INT4 FlatQuant, ADC disabled** (int4_no_adc) — INT4 FlatQuant with the ADC floor turned off. Measures the FlatQuant transform quality alone; trained with the Signed formula so the calibration does not see ADC noise, giving WikiText-2 PPL 12.56.
- **Unsigned PTQ** (best_ptq) — INT4 FlatQuant retrained from scratch with the unsigned δ_u formula at global $k = 16$.
- **Unsigned PTQ + per-layer k search** (best_ptq_k) — the Unsigned PTQ checkpoint above with per-layer k chosen in-place from the calibration statistics, without retraining FlatQuant.
- **Unsigned PTQ + per-layer k + FlatQuant retraining** (best_ptq_k_recal) — the same per-layer k vector as the previous row, followed by retraining FlatQuant with those k values baked in; tests whether retrained transforms improve over the in-place search.
- **Unsigned PTQ + post-ADC LoRA** (best_lora) — the Unsigned PTQ checkpoint plus a post-ADC residual LoRA branch ($r = 4$, all seven projections, CE+KL loss, $\lambda = 0.5$, temperature 2.0, 5 epochs at 10^{-4}).
- **Unsigned PTQ + per-layer k + post-ADC LoRA** (best_lora_k) — the same LoRA branch trained on top of the per-layer k checkpoint. This is the final configuration of the Unsigned series.

5.5.5 Experiment History and Hardware Setting Evolution

The Unsigned configuration adopted above is the endpoint of two earlier hardware-setting iterations. They are documented here so that the choice of the final configuration is not arbitrary; Table 13 summarises the comparison before the per-layer k search and LoRA correction are applied.

Table 13: Hardware-setting evolution on Llama-3.2-1B, staged PTQ baseline only (no per-layer k search, no LoRA; the corresponding row in Table 14 is best_ptq). “Cost” is the number of ADC reads per tile relative to the Signed reference.

Hardware setup	δ	Wiki PPL	C4 PPL	Cost
Signed readout	≈ 6.12	26.66	45.62	$1\times$
Four-read unsigned decomposition	$\approx 3.06/\text{branch}$	18.08	28.41	$4\times$
Single-read unsigned shift-subtract	≈ 14.06	39.00	67.78	$1\times$

The starting point is the Signed readout studied in the previous subsections, which reads a signed accumulation $y_{\text{int}} \in [-Mq_xq_w, +Mq_xq_w]$ directly and uses the step-size formula $\delta = 2Mq_xq_w/(2^{b_a}k) \approx 6.12$ at $k = 16$. Under this setting the staged PTQ baseline reaches WikiText-2 perplexity 26.66 and post-ADC LoRA reaches 14.03 — the reference numbers for any hardware that can perform a signed analog accumulation directly. In practice, however, most physically realisable optical and in-memory crossbars accept only non-negative inputs, so the Signed readout is best read as an idealised reference rather than

a deployable target.

A first attempt to bridge to non-negative hardware was the four-read unsigned decomposition, in which the signed codes are split into their positive and negative parts,

$$\hat{x} = \hat{x}^+ - \hat{x}^-, \quad \hat{w} = \hat{w}^+ - \hat{w}^-, \quad \hat{x}^\pm, \hat{w}^\pm \geq 0, \quad (53)$$

so that the signed inner product decomposes into four products of non-negative tensors,

$$\langle \hat{x}, \hat{w} \rangle = \langle \hat{x}^+, \hat{w}^+ \rangle + \langle \hat{x}^-, \hat{w}^- \rangle - \langle \hat{x}^+, \hat{w}^- \rangle - \langle \hat{x}^-, \hat{w}^+ \rangle. \quad (54)$$

Each of the four terms is a non-negative MVM that can be executed on the crossbar directly; the signed result is then assembled digitally. The branchwise step size $\delta_{\text{branch}} \approx 3.06$ is twice as fine as the Signed reference, and the staged PTQ accordingly improved to 18.08 WikiText-2 PPL (28.41 on C4). The catch is that each layer now requires $4\times$ as many ADC reads, which roughly quadruples the dominant area-and-energy cost of the analog path and renders this option uncompetitive even though the perplexity is excellent.

The final iteration is the single-read unsigned shift-subtract scheme used in the rest of this section: both code tensors are lifted to $[0, 2^b - 1]$ via the zero-point shift, a single non-negative MVM is executed on the hardware, and the deterministic zero-point terms are subtracted digitally, while the ADC rounding and clipping error remains as described in Section 5.5.1. The MVM cost returns to one ADC read per tile, and FlatQuant is retrained from scratch with the matching unsigned δ_u formula so that calibration and deployment use the same δ throughout. Table 14 summarises the results. At global $k = 16$ the Unsigned PTQ baseline gives 39.00/67.78 on Wiki / C4 — coarser than the Signed reference, as expected from the factor-of-two-larger δ_u — but the rest of the toolbox is then available. Adding the per-layer k search (Unsigned PTQ + per-layer k) lowers this to 21.64/33.79; the post-ADC LoRA on top of the global- k checkpoint (Unsigned PTQ + post-ADC LoRA) reaches 17.29/29.26; and combining both (Unsigned PTQ + per-layer k + post-ADC LoRA) gives the final **14.56/23.78** — within striking distance of the idealised Signed LoRA reference without increasing the number of analog MVM / ADC reads, aside from the small digital LoRA overhead reported in Table 9. Re-training FlatQuant on the per-layer k values found by the search (Unsigned PTQ + per-layer k + FlatQuant retraining, 22.19/36.94) is in fact slightly worse than the in-place search: fitting the transforms to a specific k vector over-specialises them and hurts generalisation, so the in-place variant is preferred.

Table 14: Unsigned shift-subtract readout results on Llama-3.2-1B. $\delta_u \approx 14.1$ at global $k = 16$.

Configuration	Short ID	Wiki PPL	C4 PPL
bfloat16 baseline	fp	8.68	13.13
INT4 FlatQuant, ADC disabled	int4_no_adc	12.56	20.13
Unsigned PTQ ($k = 16$)	best_ptq	39.00	67.78
Unsigned PTQ + per-layer k	best_ptq_k	21.64	33.79
Unsigned PTQ + per-layer k + FQ retrain	best_ptq_k_recal	22.19	36.94
Unsigned PTQ + post-ADC LoRA	best_lora	17.29	29.26
Unsigned PTQ + per-layer k + post-ADC LoRA	best_lora_k	14.56	23.78

6 Discussion

The experiments in Sec. 5 suggest that the analog–digital boundary should be treated as part of the model compression problem, not merely as a hardware implementation detail. The ADC changes the error model of LLM quantization: it adds a post-accumulation output lattice whose resolution is fixed by the hardware and cannot be freely adapted per layer or per token. This section discusses the main implications of this observation.

6.1 ADC Error Is Not the Same as Digital Quantization Error

A central lesson of the results is that ordinary low-bit PTQ quality does not predict analog deployment quality. The no-ADC setting measures the error introduced by quantizing weights and activations. The ADC setting measures an additional error source: the discretization of the analog tile output. These two errors interact, but they are not the same.

This distinction explains why a model can appear usable under digital INT4 evaluation and still fail once the ADC is enabled. The ADC is applied after accumulation, where information from many input channels has already been mixed into one analog value. If this value is rounded to the same ADC code as another distinct accumulation, the downstream network cannot recover the difference. Thus, the ADC creates a failure mode that is absent from standard digital PTQ: the model is not only approximating weights and activations, but also losing resolution at every analog output.

The practical consequence is that no-ADC perplexity is only a diagnostic metric. It is useful for separating ordinary quantization error from ADC-induced error, but it is not sufficient as a deployment criterion. Any PTQ pipeline for analog inference must evaluate the ADC-on model end to end.

6.2 ADC-Aware Calibration Reduces a Distribution Mismatch

Standard FlatQuant calibration is block-wise: each block is optimized using clean teacher activations. This is well matched to digital PTQ, but not to analog inference. During analog deployment, the input to a block is already affected by the ADC outputs of all previous blocks. The block therefore sees a different distribution at inference than it saw during calibration.

Cross-block propagation addresses this mismatch. Its role is not to add capacity to the model, but to make the calibration signal more faithful to the deployment signal. The student is calibrated on the kind of activations it will actually receive after earlier ADC steps, so error accumulation becomes visible during PTQ.

The mixed objective is needed because matching deployment too aggressively is also harmful. ADC-propagated inputs contain a noisy and non-smooth error signal, especially in INT4. Clean reconstruction provides a stable target, while the ADC-aware term teaches robustness to the hardware quantizer. The final calibration objective therefore reflects a trade-off: enough clean signal to learn useful transformations, and enough ADC signal to avoid overfitting to an idealized digital path.

6.3 Outliers Turn ADC Quantization into a Range-Allocation Problem

The ADC problem is especially severe for LLMs because their activation distributions are highly non-uniform across channels. A few channels can have much larger magnitude than the rest, but these channels cannot simply be clipped away because they may carry important information. The ADC must therefore allocate its finite range between preserving outliers and resolving the bulk of the distribution.

This reframes rotations and diagonals as range-allocation tools. Rotations redistribute activation energy across channels, making the signal less concentrated. Diagonal scaling adds per-channel control that rotations alone cannot provide. In the ADC setting, these transformations are useful not only because they improve digital quantization, but because they shape how the analog accumulation lands on the fixed ADC lattice.

The diagonal ablations suggest that different parts of the transformer serve different roles. MLP-side scaling mainly improves the quality of the low-bit transform itself, while attention-side scaling more directly affects robustness to ADC noise. This explains why staged training is useful: it separates the clean distribution-flattening problem from the ADC-robustness problem instead of forcing one joint objective to solve both from the start.

6.4 Why Pure PTQ Has a Natural Ceiling

The plateau observed in pure PTQ is not just an optimization artifact. All FlatQuant-style transformations act before the ADC. They can reshape the pre-ADC signal so that it uses the available ADC range more effectively, but they cannot undo the discretization after it has happened.

Once the ADC maps a continuous accumulation to a finite code, all sub- δ variation inside that bin is removed from the frozen analog path. A better pre-ADC transform can reduce the frequency of bad roundings, dead-zone hits, or saturation events, but it cannot represent corrections that must be applied after the output code has already been chosen.

This is the structural reason why ADC-aware PTQ is necessary but not sufficient. It can turn a collapsed model into a usable model, but in the low-bit regime it eventually reaches a hardware-defined floor. To move beyond that floor, the correction has to operate after the ADC or the hardware lattice itself has to change.

6.5 Post-ADC Correction Works Because It Is Placed After the Lossy Step

The post-ADC LoRA branch is effective because it is placed on the digital side of the analog–digital boundary. A correction placed before the ADC is still subject to the same output lattice and can be rounded away. A correction placed after the ADC can directly adjust the value that the next digital part of the network receives.

This makes the post-ADC branch qualitatively different from another pre-ADC transform. It does not try to make the analog accumulation easier to quantize. Instead, it learns a residual compensation for the frozen quantized path after the ADC has already removed information. The low-rank parameterization keeps this correction small, while teacher distillation helps it match the full-precision model beyond the next-token labels alone.

The C4 improvement is important in this interpretation. Since the adapter is trained on WikiText-2 but also improves held-out C4 perplexity, the branch is not merely memorizing the calibration text. It appears to learn a systematic correction for the ADC-distorted representations.

6.6 Hardware Setting Matters

The comparison between the Signed and Unsigned readouts highlights that ADC-aware PTQ cannot be separated from the hardware readout model. Changing the ADC range, sign representation, or scaling factor changes the effective output lattice and therefore changes the calibration problem.

The Unsigned experiments show the trade-off clearly. A four-read unsigned decomposition can improve accuracy by using more favorable non-negative branches, but it increases the number of ADC reads. The single-read unsigned shift-subtract formulation restores the one-read-per-tile cost, but uses a coarser effective step size and therefore requires additional calibration support. Per-layer k search is useful in this setting because it adapts the ADC scale to the distribution of each projection without changing the model weights.

The broader implication is that software PTQ and hardware ADC design should not be optimized independently. The ADC step size, tile size, scaling factor, and calibration objective jointly determine whether the analog output lands in a useful part of the ADC range.

6.7 Practical Implications

The results suggest four practical guidelines for analog LLM deployment.

First, ADC-on evaluation should be part of the compression pipeline. A no-ADC result can diagnose digital quantization quality, but it can hide the dominant hardware-induced error.

Second, calibration should reproduce the inference signal path. If blocks receive ADC-corrupted activations at inference, they should not be calibrated only on clean teacher activations.

Third, activation outliers should be treated as an ADC-range problem. Rotations and diagonal scaling are useful because they redistribute signal energy before the analog readout, not only because they reduce a reconstruction loss.

Fourth, a small digital correction can be a favorable compromise between pure PTQ and full hardware-aware QAT. It adds limited digital overhead while avoiding end-to-end retraining of the full LLM.

6.8 Limitations

The conclusions above should be interpreted with several limitations.

Simulation rather than silicon. All experiments use a software ADC simulator. Real optical or in-memory hardware may introduce additional non-idealities, including thermal

noise, drift, non-uniform ADC bins, offset error, and temperature dependence. The simulator isolates the finite-resolution ADC effect, but it does not replace validation on a physical accelerator.

Single model scale. The full evaluation is performed on Llama-3.2-1B. The method is designed for decoder-only LLMs, but larger models may have different activation outlier statistics and may require different calibration budgets. Scaling to larger checkpoints is also affected by batch-size limits and optimization noise during ADC-aware calibration.

Calibration distribution. FlatQuant transformations and LoRA adapters are calibrated on WikiText-2. C4 provides evidence of out-of-domain transfer, but it does not cover the full diversity of deployment data. Domain shifts may change the activation distributions and therefore the optimal ADC-aware transform.

Simplified ADC model. The main experiments assume a uniform ADC lattice with a fixed step size. Real ADCs can have non-uniform bin spacing, offset, hysteresis, latency, and layer-dependent drift. The per-layer k experiments explore one form of configurability, but they do not exhaust the hardware design space.

Digital overhead of post-ADC correction. The LoRA correction is small, but it is not free. It requires a digital branch and access to the activation used by the adapter. Whether this correction should run on a digital co-processor, be restricted to selected projections, or be integrated into other post-processing depends on the final accelerator design.

7 Conclusion

This thesis studied post-training quantization for Llama-3.2-1B under analog in-memory inference with finite-resolution ADCs. The central question was whether an already trained LLM can be adapted to a fixed hardware ADC without full quantization-aware retraining. The answer is qualified: ADC-aware PTQ is necessary and highly effective, but it plateaus in the low-bit regime; a small post-ADC digital correction is needed to recover much of the remaining loss.

RQ1: How much quality degradation is caused specifically by ADC output quantization? The ADC is a dominant error source, not a minor implementation detail. At INT8, the digital W8A8 path is essentially lossless (WikiText-2 PPL 8.69 vs. the bfloat16 baseline of 8.68), but enabling the ADC moves the same checkpoint to 16.85. The pattern is stronger at INT4: the digital INT4 PTQ baseline reaches Wiki PPL 14.13, while the same precision under ADC reaches 26.66 even after ADC-aware calibration. Without any ADC-aware mitigation the collapse is catastrophic (2354.86 on the INT4 baseline). ADC output quantization therefore introduces a separate failure mode that is invisible to standard digital PTQ.

RQ2: To what extent do FlatQuant-style transforms mitigate ADC degradation? Transform-based PTQ recovers most of the collapse. Block-wise FlatQuant with propagated ADC-quantized inputs, an α -mixed objective, bounded learnable diagonals, and

staged training reduces the INT4 ADC PPL from 2354.86 (naive baseline) to 26.66 on WikiText-2, and reaches 45.62 on the held-out C4 evaluation. This is a $\times 88$ recovery on WikiText-2 and shows that learned pre-ADC transformations are an essential first step for analog LLM inference. At the same time, 26.66 remains $\approx 3.1\times$ worse than the bfloat16 reference, exposing a hardware-induced plateau that pre-ADC transforms alone cannot cross.

RQ3: Are ADC-aware objectives required, or is clean reconstruction sufficient?

Explicit ADC-aware objectives are required. Cross-block propagation of ADC-quantized inputs and the α -mixed loss both outperform clean reconstruction. In the INT8 calibration ablation (Section 5.4.1, 128 samples, WikiText-only) the propagated objective alone moves ADC PPL from 28.86 (clean reconstruction) to 14.55, and adding the bin-center auxiliary loss reduces it further to 14.41 — a $\times 2.0$ drop relative to the clean baseline. In the INT4 series an α sweep selects $\alpha = 0.5$, which gives a much better trade-off than either $\alpha = 0$ (clean) or $\alpha = 1$ (full propagation) — the noise from full ADC propagation is too aggressive at INT4 but a balanced mixture exposes the calibration to ADC error without destabilising it.

RQ4: Which projections and layers are most sensitive to ADC quantization?

Sensitivity is heterogeneous across projections, and this heterogeneity is exploitable rather than uniform noise. Two pieces of evidence support this. First, the staged-training ablation (Section 5.4.4) shows that MLP-side and attention-side diagonal scalings serve qualitatively different roles: MLP diagonals primarily improve the quality of the low-bit transform itself, while attention diagonals more directly affect robustness to ADC noise. Joint training of the two destabilises attention; staging them recovers performance. Second, the per-layer k search in the Unsigned series (Section 5.5) shows that different projections benefit from different ADC scaling factors — selecting k in place per projection lowers the Unsigned PTQ Wiki PPL from 39.00 to 21.64 without retraining the transforms. Both observations indicate that ADC sensitivity is not uniformly distributed across the architecture.

RQ5: When is lightweight adaptation beyond PTQ required?

It is required as soon as the pure PTQ plateau is reached. A rank-4 post-ADC residual LoRA branch, trained with cross-entropy plus KL distillation from the bfloat16 teacher, reduces WikiText-2 ADC PPL from 26.66 to 14.03 and C4 ADC PPL from 45.62 to 23.20, while adding only 0.228% trainable parameters. The C4 improvement is important because LoRA is trained on WikiText-2 only, so the correction is not memorising the calibration text. Placement is critical: a sister adapter placed *before* the ADC is itself quantized by the fixed output lattice and fails in the ablation. A correction placed after the ADC operates in the digital domain and compensates for residual information that the frozen analog path has already removed. Thus, post-ADC LoRA is not another PTQ transform; it is an output-side residual adaptation targeted specifically at the analog–digital boundary.

Overall conclusion. Analog optical and in-memory accelerators promise more energy-efficient LLM inference, but their ADCs introduce an additional quantization problem that standard digital PTQ does not solve. The results of this thesis show that the model must be adapted to the ADC in two stages. First, ADC-aware PTQ reshapes the activations and weights so that the analog output uses the fixed ADC range more effectively. Second,

a lightweight digital post-ADC correction compensates for the residual error that cannot be removed before the ADC.

The main message is therefore:

ADC-aware PTQ makes analog low-bit LLM inference usable, but post-ADC digital correction is needed to move beyond the PTQ plateau.

Future work. Several directions remain open. First, the method should be scaled to larger LLMs and evaluated under realistic batch-size constraints. Second, the software ADC simulator should be extended with additional device-level effects and validated against silicon or optical hardware measurements. Third, hardware–software co-design could tune ADC step size, tile size, and per-layer scaling jointly with the PTQ objective. Fourth, the post-ADC correction could be made more selective, applying it only to the most ADC-sensitive projections or layers to reduce digital overhead further. Finally, future work could explore objectives that target sub- δ output error directly, rather than using only reconstruction losses before the ADC.

Overall, the thesis shows that the analog-digital boundary is an algorithmic object, not just a hardware detail. Treating the ADC as part of the quantization pipeline is essential for making low-bit LLMs work on analog accelerators.

References

- [1] Yamato Arai and Yuma Ichikawa. “Quantization Error Propagation: Revisiting Layer-Wise Post-Training Quantization”. In: *arXiv preprint arXiv:2504.09629* (2025).
- [2] Saleh Ashkboos et al. “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs”. In: *arXiv preprint arXiv:2404.00456* (2024).
- [3] Yonatan Bisk et al. “PIQA: Reasoning about Physical Commonsense in Natural Language”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2020.
- [4] Yelysei Bondarenko, Riccardo Del Chiaro, and Markus Nagel. “Low-Rank Quantization-Aware Training for LLMs”. In: *arXiv preprint arXiv:2406.06385* (2024).
- [5] Julian Büchel et al. “Analog Foundation Models”. In: *arXiv preprint arXiv:2505.09663* (2025).
- [6] Yuji Chai et al. “INT2.1: Towards Fine-Tunable Quantized Large Language Models with Error Correction through Low-Rank Adaptation”. In: *arXiv preprint arXiv:2306.08162* (2023).
- [7] Mengzhao Chen et al. “EfficientQAT: Efficient Quantization-Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2407.11062* (2024).
- [8] Jungwook Choi et al. “PACT: Parameterized Clipping Activation for Quantized Neural Networks”. In: *arXiv preprint arXiv:1805.06085* (2018).
- [9] Christopher Clark et al. “BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2019.
- [10] Peter Clark et al. “Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge”. In: *arXiv preprint arXiv:1803.05457* (2018).
- [11] Tim Dettmers et al. “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”. In: *arXiv preprint arXiv:2208.07339* (2022).
- [12] Xin Ding et al. “CBQ: Cross-Block Quantization for Large Language Models”. In: *arXiv preprint arXiv:2312.07950* (2023).
- [13] Abhimanyu Dubey et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024). URL: <https://arxiv.org/abs/2407.21783>.
- [14] Steven K. Esser et al. “Learned Step Size Quantization”. In: *arXiv preprint arXiv:1902.08153* (2019).
- [15] Leo Gao et al. *A Framework for Few-Shot Language Model Evaluation*. <https://github.com/EleutherAI/lm-evaluation-harness>. 2024.
- [16] Amir Gholami et al. “A Survey of Quantization Methods for Efficient Neural Network Inference”. In: *arXiv preprint arXiv:2103.13630* (2021).
- [17] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding”. In: *International Conference on Learning Representations (ICLR)*. 2021.
- [18] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2022. URL: <https://arxiv.org/abs/2106.09685>.
- [19] Benoit Jacob et al. “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference”. In: *arXiv preprint arXiv:1712.05877* (2018).

- [20] Manuel Le Gallo et al. “AnalogNAS: A Neural Network Design Framework for Accurate Inference with Analog In-Memory Computing”. In: *arXiv preprint arXiv:2305.10459* (2023).
- [21] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration”. In: *arXiv preprint arXiv:2306.00978* (2023).
- [22] Haotian Liu et al. “LLM-QAT: Data-Free Quantization Aware Training for Large Language Models”. In: *arXiv preprint arXiv:2305.17888* (2023).
- [23] Ruikang Liu et al. “FlatQuant: Flatness Matters for LLM Quantization”. In: *arXiv preprint arXiv:2410.09426* (2024).
- [24] Zechun Liu et al. “SpinQuant: LLM Quantization with Learned Rotations”. In: *arXiv preprint arXiv:2405.16406* (2024).
- [25] Stephen Merity et al. “Pointer Sentinel Mixture Models”. In: *International Conference on Learning Representations (ICLR)*. 2017. URL: <https://arxiv.org/abs/1609.07843>.
- [26] Todor Mihaylov et al. “Can a Suit of Armor Conduct Electricity? A New Dataset for Open Book Question Answering”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2018.
- [27] Markus Nagel et al. “Up or Down? Adaptive Rounding for Post-Training Quantization”. In: *arXiv preprint arXiv:2004.10568* (2020).
- [28] Reiner Pope et al. “Efficiently Scaling Transformer Inference”. In: *Proceedings of Machine Learning and Systems 5* (2023).
- [29] Colin Raffel et al. “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”. In: *Journal of Machine Learning Research* 21.140 (2020), pp. 1–67. URL: <https://arxiv.org/abs/1910.10683>.
- [30] Keisuke Sakaguchi et al. “WinoGrande: An Adversarial Winograd Schema Challenge at Scale”. In: *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 2021.
- [31] Meyer Scetbon and James Hensman. “Low-Rank Correction for Quantized LLMs”. In: *arXiv preprint arXiv:2412.07902* (2024).
- [32] Wenqi Shao et al. “OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models”. In: *arXiv preprint arXiv:2308.13137* (2023).
- [33] James Spall, Xianxin Guo, and A. I. Lvovsky. “Hybrid Training of Optical Neural Networks”. In: *Optica* 9.7 (2022), pp. 803–811. URL: <https://arxiv.org/abs/2203.11207>.
- [34] Mingxue Tang et al. “RAOQ: Reshape and Adapt for Output Quantization”. In: *International Conference on Machine Learning (ICML)*. 2024. URL: <https://openreview.net/forum?id=8mKXMnhnFW>.
- [35] Xiuying Wei et al. “Outlier Suppression+: Accurate Quantization of Large Language Models by Equivalent and Optimal Shifting and Scaling”. In: *arXiv preprint arXiv:2304.09145* (2023).
- [36] Xiuying Wei et al. “QDrop: Randomly Dropping Quantization for Extremely Low-bit Post-Training Quantization”. In: *arXiv preprint arXiv:2203.05740* (2022).
- [37] Hao Wu et al. “Integer Quantization for Deep Learning Inference: Principles and Empirical Evaluation”. In: *arXiv preprint arXiv:2004.09602* (2020).

- [38] Guangxuan Xiao et al. “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models”. In: *arXiv preprint arXiv:2211.10438* (2022).
- [39] Patrick Xiao et al. “On the Accuracy of Analog Neural Network Inference Accelerators”. In: *arXiv preprint arXiv:2109.01262* (2021).
- [40] Zhewei Yao et al. “ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers”. In: *arXiv preprint arXiv:2206.01861* (2022).
- [41] Rowan Zellers et al. “HellaSwag: Can a Machine Really Finish Your Sentence?” In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2019.
- [42] Cheng Zhang et al. “LQER: Low-Rank Quantization Error Reconstruction for LLMs”. In: *arXiv preprint arXiv:2402.02446* (2024). URL: <https://arxiv.org/abs/2402.02446>.

A Key Code Listings

This appendix collects the four code components that implement the method described in Sec. 4. All listings are simplified for readability; the complete reference implementation is available at <https://github.com/BurykinaA/quantization-techniques> under ADC/llama/core/ (full pipeline, see Sec. 4) and ADC/best/core/ for the cleaned-up reproduction code. Throughout these listings, s_w_vec denotes the per-channel weight scale (shape $[out_features]$) computed once at calibration time as

$$s_w_vec[c] = \frac{1}{q_w} \max_j |W_{c,j}|.$$

A.1 ADC Forward Pass

Listing 1 shows the per-tile ADC forward pass. It corresponds to `QATLinearADC.forward` in `adc_layers.py` and implements Eq. (21) together with the dead-zone and clipping behaviour: per-token activation scale, per-channel weight scale, integer MVM in code domain, ADC floor + clamp, and dequantization. The step size δ is fixed at construction from (M, q_x, q_w, b_a, k) .

```

1 def forward(self, x):
2     # Per-token activation scale s_x (amax over feature dim)
3     s_x = x.abs().amax(dim=-1, keepdim=True).clamp(min=1e-6) / act_levels
4     code_x = round_ste(x / s_x).clamp(qmin_x, qmax_x)
5
6     # Per-channel weight scale s_w
7     code_w = round_ste(safe_divide(self.weight, s_w_vec.view(-1, 1)))
8     code_w = torch.clamp(code_w, qmin_w, qmax_w)
9
10    # Integer MVM in code domain
11    y_int = F.linear(code_x, code_w, bias=None)
12
13    # ADC: floor + clamp. delta = (2 * M * q_x * q_w) / (2^b_a * k)
14    if self.bypass_adc:
15        adc_output = y_int
16    else:
17        y_adc_codes = floor_ste(y_int / self.delta)
18        y_adc_codes = torch.clamp(y_adc_codes, self.na, self.pa)
19        adc_output = y_adc_codes * self.delta
20
21    # Dequantise: s_x [B,1] * s_w_vec [out] -> [B, out]
22    y_real = adc_output * s_x * s_w_vec
23    if self.bias is not None:
24        y_real = y_real + self.bias
25    return y_real

```

Listing 1: Per-tile ADC forward pass (`QATLinearADC.forward`).

`TiledLinearADC` wraps several `QATLinearADC` tiles side-by-side along the input dimension whenever the projection width exceeds the crossbar size M (the `mvm_limit`). Each tile has its own δ derived from its local in-features.

A.2 Kronecker Transform with Diagonal

Listing 2 shows the trainable Kronecker transform $T = T_1 \otimes T_2$ together with the optional per-channel diagonal D (file `flat_quant.py`, class `KroneckerTransform`). Each

Kronecker factor is parameterized as $U \text{diag}(s) V^\top$ with U, V kept orthogonal via PyTorch's `matrix_exp` parametrisation, which preserves orthogonality under unconstrained gradient updates. The `add_diag` branch implements $\tilde{T} = T \cdot D$ from Section 4.

```

1 def forward(self, x, inv_t=False):
2     # Optional per-channel diagonal D (Sec. \ref{sec:diagonals}).
3     # The clamp here only guards against numerical zero;
4     # the working bound d_i in [1e-4, 10] is enforced separately
5     # via the optimiser projection in Listing \ref{lst:diag_proj}.
6     if self.add_diag and self.use_diag:
7         d = self.diag_scale.abs().clamp(min=1e-8)
8         x = x / d if inv_t else x * d
9
10    # Left and right Kronecker factors T_i = U_i @ diag(s_i) @ V_i.T
11    dl = self.diag_left.abs().clamp(min=1e-6)
12    dr = self.diag_right.abs().clamp(min=1e-6)
13    if inv_t:
14        dl, dr = 1.0 / dl, 1.0 / dr
15    mat_l = self.u_left.weight @ torch.diag(dl) @ self.v_left.weight.T
16    mat_r = self.u_right.weight @ torch.diag(dr) @ self.v_right.weight.T
17
18    # x_resaped @ (T_1 (x) T_2) = T_1^T reshape T_2 reshape
19    return _kronecker_matmul(x, mat_l, mat_r)

```

Listing 2: Kronecker transform forward with optional diagonal scaling (`KroneckerTransform.forward`). The `clamp(min=...)` calls in this listing only ensure positivity / numerical safety; the working bounds on the trainable diagonals are enforced as an optimiser projection after every step (see Listing 3).

```

1 optimizer.step()
2 with torch.no_grad():
3     for name, param in layer.named_parameters():
4         if "diag_left" in name or "diag_right" in name:
5             param.data.clamp_(min=0.1) # internal SVD-style
6             # factor scales
7         elif "diag_scale" in name:
8             param.data.clamp_(min=1e-4, max=10.0) # external
9             # per-channel D

```

Listing 3: Bounded-diagonal projection step in `calibrate_flat_quant`, applied after every `optimizer.step()`. This is where the working bounds on `diag_scale` (range 1e-4 to 10) and `diag_left/diag_right` (minimum 0.1) are actually enforced.

At deployment, `to_eval_mode` pre-computes and caches the full forward, inverse, and transpose matrices and discards the parametrised U, V tensors. The transforms are then folded into the linear weights via `reparameterize`, leaving zero inference overhead.

A.3 Block-Level Loss with α -Mixing

Listing 4 shows the block-level reconstruction loss with the dual-forward α -mixed objective from Section 4. The same block runs twice: once on the clean FP teacher input x^* and once on the ADC-quantized output $\hat{x}$ from the previous block, and the two losses are combined linearly.

```

1 # y_star: teacher block output on clean FP input (no grad)
2 # x_clean: clean FP activations
3 # x_adc: ADC-quantized activations from previous block (propagation)
4 y_fp = block(x_clean) # clean forward through quantized
    block
5 y_adc = block(x_adc) # propagated forward (sees real ADC
    noise)
6
7 loss_fp = F.mse_loss(y_fp, y_star)
8 loss_adc = F.mse_loss(y_adc, y_star)
9
10 # Eq. \eqref{eq:alpha_mix}: L = (1 - alpha) * L_FP + alpha * L_ADC
11 loss = (1.0 - alpha) * loss_fp + alpha * loss_adc

```

Listing 4: Block-level α -mixed reconstruction loss.

Staged training (Section 4.1.5) is implemented by simply varying which parameters require gradients and which α is used per stage:

```

1 # Stage A: 30 epochs, lr = eta, alpha = 0 (clean-input loss)
2 for p in layer.parameters(): # freeze everything
3     p.requires_grad_(False)
4 for p in kron_params_all_projections: # Kron factors of q,k,v,o, gate,
    up, down
5     p.requires_grad_(True) # -> always trainable
6 for p in mlp_diag_scale_params: # MLP diagonal D^(m)
7     p.requires_grad_(True) # -> trained in Stage A
8 # attention diag_scale D^(a) stays frozen (requires_grad = False)
9 train(epochs=30, lr=eta, alpha=0.0)
10
11 # Stage B: 10 epochs, lr = 0.1 * eta, alpha = 0.5 (mixed FP /
    ADC-propagated loss)
12 for p in layer.parameters(): # freeze everything again
13     p.requires_grad_(False)
14 for p in kron_params_all_projections: # Kron factors trainable again,
15     p.requires_grad_(True) # but at 10x lower lr they are
    only fine-tuned
16 for p in attn_diag_scale_params: # attention diagonal D^(a)
17     p.requires_grad_(True) # -> trained in Stage B
18 # MLP diag_scale D^(m) stays frozen at its Stage-A value
19 train(epochs=10, lr=0.1 * eta, alpha=0.5)

```

Listing 5: Staged training schedule (Stage A then Stage B). At the start of each stage all parameters are frozen and the trainable subset is re-enabled. Kronecker factors of all seven projections are optimised in both stages; the staging only toggles which per-channel diagonal (MLP vs attention) is trained, and Stage B drops the learning rate by 10x so the MLP basis is fine-tuned, not displaced.

A.4 Post-ADC Residual LoRA

Listing 6 shows the post-ADC residual LoRA wrapper. The wrapper is implemented in `adc_lora.py` as class `ResidualLoRATiledLinearADC`. The frozen `TiledLinearADC` produces the quantized path; a parallel rank- r branch reads the FP input and adds its float32 output to the ADC result. `lora_B` is initialized to zero so the wrapped layer initially matches the frozen PTQ checkpoint exactly.

```

1 class ResidualLoRATiledLinearADC(nn.Module):
2     def __init__(self, tiled_layer, r=4, alpha=8.0):
3         super().__init__()
4         self.tiled_layer = tiled_layer          # frozen ADC path
5         self.scaling = alpha / r
6         self.lora_A = nn.Linear(tiled_layer.in_features_total, r,
7                                 bias=False, dtype=torch.float32)
8         self.lora_B = nn.Linear(r, tiled_layer.out_features,
9                                 bias=False, dtype=torch.float32)
10        nn.init.kaiming_uniform_(self.lora_A.weight, a=math.sqrt(5))
11        nn.init.zeros_(self.lora_B.weight)      # zero start
12        for p in self.tiled_layer.parameters():
13            p.requires_grad_(False)             # freeze base path
14
15    def forward(self, x):
16        y_base = self.tiled_layer(x)            # frozen ADC path
17        y_lora = self.lora_B(self.lora_A(x.float())) * self.scaling
18        return y_base + y_lora.to(y_base.dtype)

```

Listing 6: Post-ADC residual LoRA wrapper (ResidualLoRATiledLinearADC).

Pre-ADC placement is implemented by the sister class PreADCLoRATiledLinearADC, which adds the rank- r update to the weights of each tile *before* integer quantization and ADC. As shown in Section 5, this placement diverges (ADC PPL ~ 105) and is included only as an ablation.